

Energy-Efficient Trajectory Optimization With Wireless Charging in UAV-Assisted MEC Based on Multi-Objective Reinforcement Learning

Fuhong Song¹, Mingsen Deng¹, *Member, IEEE*, Huanlai Xing², *Member, IEEE*, Yanping Liu³, Fei Ye⁴,
and Zhiwen Xiao¹

Abstract—This paper investigates the problem of energy-efficient trajectory optimization with wireless charging (ETWC) in an unmanned aerial vehicle (UAV)-assisted mobile edge computing system. A UAV is dispatched to collect computation tasks from specific ground smart devices (GSDs) within its coverage while transmitting energy to the other GSDs. In addition, a high-altitude platform with a laser beam is deployed in the stratosphere to charge the UAV, so as to maintain its flight mission. The ETWC problem is characterized by multi-objective optimization, aiming to maximize both the energy efficiency of the UAV and the number of tasks collected via optimizing the UAV's flight trajectories. The conflict between the two objectives in the problem makes it quite challenging. Recently, some single-objective reinforcement learning (SORL) algorithms have been introduced to address the aforementioned problem. Nevertheless, these SORLs adopt linear scalarization to define the user utility, thus ignoring the conflict between objectives. Furthermore, in dynamic MEC scenarios, the relative importance assigned to each objective may vary over time, posing significant challenges for conventional SORLs. To solve the challenge, we first build a multi-objective Markov decision process that has a vectorial reward mechanism. There is a corresponding relationship between each component of the reward and one of the two objectives. Then, we propose a new trace-based experience replay scheme to modify sample efficiency and reduce replay buffer bias, resulting in a modified multi-objective reinforcement learning algorithm. The experiment results validate that the proposed algorithm can obtain better adaptability to dynamic preferences and a more favorable balance between objectives compared with several algorithms.

Index Terms—Mobile edge computing, multi-objective reinforcement learning, trajectory optimization, unmanned aerial vehicle, wireless charging.

I. INTRODUCTION

WITH the fast development of mobile communication technology, ground smart devices (GSDs) play a significant role in diverse applications, such as intelligent grazing facilitated by high-definition cameras and environmental monitoring utilizing meteorological sensors. [1]. Specifically, GSDs can be distributed to monitor and sense their surroundings, thus offering unprecedented opportunities for emerging intelligent applications, such as augmented reality and infrastructure monitoring. These applications are usually computing-intensive, resulting in a surging demand for computing resources. However, GSDs often suffer from limited computing capability and battery life, due to small physical sizes and stringent production cost constraints [2].

Mobile edge computing (MEC) has been introduced to enhance the computing capability of GSDs by relocating computing resources to the network edge. Within this paradigm, GSDs are empowered to offload their computing-intensive applications to ground base stations (GBSs) situated nearby, thereby enabling them to efficiently support such applications [3]. In general, traditional MEC technologies are based on GBSs, which are placed on the ground and kept fixed. This means the wireless communication coverage of a GBS is limited, which restricts its ability to connect to users located beyond the GBS's coverage area. Particularly, GBSs may encounter damage due to military attacks or natural disasters, causing a scarcity of computing resources and a decline in offloading performance [4].

Unmanned aerial vehicles (UAVs) have emerged as an efficient solution for extending communication coverage and enhancing deployment efficiency due to their high mobility and excellent maneuverability [5], [6]. For example, UAVs can provide vital communication support when GBS-based infrastructures are either unavailable during disaster rescue operations or sparsely distributed in remote mountainous areas. Moreover, when there is a sudden surge in GSDs that exceeds the capacity of network services, like during a crowded football match, UAVs can serve these GSDs as emergency computing platforms.

Manuscript received 6 November 2023; revised 2 February 2024; accepted 29 March 2024. Date of publication 3 April 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62061007, in part by Guizhou Provincial Science and Technology Project under Grant ZK[2024]693 and Grant ZK[2023]028, in part by Guizhou Provincial Key Technology R&D Program under Grant QKHZC[2023]YB185, and in part by the Guizhou University of Finance and Economics Introduced Talent Research Start-Up under Grant 2022YJ039. Recommended for acceptance by H. Yao. (*Corresponding authors: Fuhong Song; Yanping Liu.*)

Fuhong Song and Mingsen Deng are with the School of Information, Guizhou University of Finance and Economics, Guiyang 550025, China (e-mail: fhsong@mail.gufe.edu.cn; msdeng@mail.gufe.edu.cn).

Huanlai Xing and Zhiwen Xiao are with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu 611756, China (e-mail: hxx@home.swjtu.edu.cn; xiao1994zw@163.com).

Yanping Liu is with the College of Big Data Statistics, Guizhou University of Finance and Economics, Guiyang 550025, China (e-mail: liuyanping6@126.com).

Fei Ye is with the Department of Computer Science, University of York, YO10 5GH York, U.K. (e-mail: fy689@york.ac.uk).

Digital Object Identifier 10.1109/TMC.2024.3384405

Therefore, equipped with communication and computing resources, UAVs can collect computation tasks from GSDs nearby and handle them.

Although UAV-assisted MEC offers various potential benefits, the energy constraint impedes its widespread use since UAVs and GSDs have limited battery life [7], [8]. Equipped with the laser beam, a high-altitude platform (HAP) can be strategically deployed in the stratosphere, making it possible for long-distance and high-power charging for UAVs [9]. However, it is unrealistic to directly charge GSDs using laser beams because it poses potential risks to both human safety and ground infrastructures. As an energy relay, a UAV can harvest the laser energy from the HAP and distribute the energy to GSDs by wireless power transfer (WPT). Moreover, the UAV can also adopt the harvested energy to process computation tasks collected by it from GSDs. In other words, the UAV plays two different roles, i.e., an energy relay and an MEC server. Nevertheless, there has been little research on the scenario in the literature.

On the other hand, simultaneously maximizing energy efficiency and the number of collected computation tasks has emerged as significant research attention in UAV-assisted MEC. The two objectives are in conflict with each other. However, most studies transform the multi-objective optimization (MOO) problem into a single-objective optimization (SOO) problem using the weighted sum method. Note that the weights assigned to each objective reflect the preferences between objectives. While the weighted sum method is adequate for situations where weights remain unchanged, it cannot be applied to scenarios with diverse preferences because they may vary over time. The dynamics further intensifies the complexity of the MOO problem, making it difficult to achieve good optimization results.

To improve the energy supply, this paper considers an HAP-and-UAV collaborative MEC system including an HAP, a UAV, and many GSDs. Laser charging and WPT technologies are employed to maintain the energy supply of the UAV and GSDs. Specifically, the HAP with a constant height floats in the stratosphere in a quasi-static manner, and it can generate a high-power laser beam to charge the UAV. The UAV can transmit the harvested energy from the HAP to GSDs within its coverage while collecting computation tasks from a specific GSD. In the proposed system, we investigate the problem of energy-efficient trajectory optimization with wireless charging (ETWC). Unlike previous studies that focus on SOO or MOO using the weighted sum method, we concentrate on the simultaneous optimization of two conflicting objectives, i.e., energy efficiency and the number of collected tasks. To adapt to the dynamics of preferences, we address the modeled MOO problem by proposing a modified multi-objective reinforcement learning (MORL) algorithm. We summarize the contributions of this paper as follows.

- We consider an HAP-and-UAV collaborative MEC system, which an HAP and a UAV cooperate with each other to provide energy supply and computing services to GSDs. The ETWC problem is formulated as an MOO problem, and its aim is to simultaneously maximize the UAV's energy efficiency and the number of collected tasks via optimizing the UAV's flight trajectories. Solving the MOO problem is quite challenging on account of the conflicting

objectives, making it difficult to strike a balance between them.

- To address the ETWC problem, we establish a multi-objective Markov decision process (MOMDP) model with a vectorial reward of two elements. Each element in the reward is associated with an optimization objective. A modified MORL algorithm with a trace-based experience replay (TER) scheme, namely MORL-TER, is proposed. The TER scheme treats a trace as an atomic unit when considering it for addition in or deletion from the experience buffer, thus can improve sample efficiency and reduce replay buffer bias.
- We perform comprehensive experiments on six generated test instances. The results demonstrate that MORL-TER can adapt to the dynamic preferences and achieve a favorable balance between objectives in most cases, as well as surpasses five state-of-the-art MORLs and two well-known multi-objective evolutionary algorithms regarding multiple evaluation indicators, including the average episodic regret, adaptation error, inverted generational distance, average energy efficiency, average number of tasks, average comprehensive objective indicator, and Friedman test.

The remainder of the paper is organized as follows. Section II provides a review of the related work in the field of UAV-assisted networks. The system model and problem formulation are presented in Section III. In Section IV, a concise overview of MOMDP is provided. Section V elaborates the proposed MORL-TER algorithm for addressing the ETWC problem. Section VI analyzes and discusses the simulation results. Finally, Section VII concludes the paper.

II. RELATED WORK

Considerable researchers have devoted significant attention to investigating diverse aspects of UAV-assisted networks with wireless charging. These investigations can primarily be classified into two categories, namely the traditional methods and reinforcement learning based methods.

A. Traditional Methods

Recently, traditional methods, such as convex optimization, heuristics, and metaheuristics, have been adopted to optimize UAV-assisted networks, yielding favorable optimization outcomes. The authors in [7] considered a UAV-assisted MEC network with wireless charging. They introduced a convex approximation method to reduce the UAV's energy consumption. The researchers in [10] investigated the trajectory and power optimization, where a GBS was deployed to charge the UAV. The concave-convex and dual decomposition based approach was introduced to maximize the energy efficiency. Hu et al. [11] studied a GBS and a UAV cooperative network, where the UAV can deliver information and energy to GSDs. They presented a block coordinate descending method to maximize the task-input bits. In [12], a multi-UAV cooperative trajectory optimization problem was studied by considering battery charging and servicing dynamic demands. A fast iterative cooperation method was introduced to plan the trajectories of UAVs. To prolong their

battery life, the authors in [13] built a non-disruptive wireless rechargeable network, where a static wireless station charger (WSC) could charge UAVs by the WPT technology. They presented a baseline method to enhance the energy efficiency of UAVs. Similarly, the researchers in [14] also investigated the energy efficiency maximization problem, and adopted the Lagrange multiplier method to address it. In [15], the task offloading and UAV charging problem was studied, and the authors in [15] developed a mixed-integer linear programming model, with the energy consumption minimized. Shi et al. [16] studied a UAV-assisted WPT system, where the UAV's energy was transmitted to GSDs. They proposed a two-stage scheme to maximize total harvested energy and minimize flight energy consumption. Wang et al. [17] investigated an air-ground integrated MEC network, where a UAV was adopted to offer offloading and charging services for GSDs. They developed a greedy algorithm to optimize the service latency. Li et al. [18] studied a UAV-assisted MEC network with wireless charging and proposed a two-stage alternating iterative algorithm to optimize the computation bits by planning the UAV's trajectories. Zhang et al. [19] proposed a UAV-assisted MEC system, where the UAV was employed to process the tasks offloaded by the GSDs. The authors developed an iterative algorithm to reduce the total energy consumption. Gu et al. [20] investigated a UAV-assisted energy-efficient MEC network, where a UAV could be charged by a GBS. They adopted the convex optimization method to decrease energy consumption and enhance the energy storage of the UAV. Yang et al. [21] investigated a UAV-assisted wireless communication network with energy harvesting, and adopted the successive convex approximation technique to lower energy consumption by optimizing flight paths. In [22], a heuristic algorithm was adopted to efficiently charge GSDs using the WPT technology. Li et al. [23] presented a multi-UAV-enabled energy harvesting network. They developed a genetic algorithm based user assignment method to improve the UAV's energy efficiency by planning flight paths.

B. Reinforcement Learning Based Methods

One of the important functions of reinforcement learning (RL) is to choose actions that can maximize the expected cumulative reward through interactions between it and the environment. Yu et al. [8] introduced a new approach by extending deep deterministic policy gradient (DDPG), which could maximize the total energy harvested by GSDs and energy efficiency of the UAV. However, the algorithm failed to adapt to the scenarios with dynamic preferences. Cheng et al. [9] investigated the task and energy offloading problem in a UAV-assisted 6 G MEC network, where an HAP with a laser beam was adopted to charge UAVs. They introduced a new approach based on multi-agent DDPG, which could improve the total system utility. Zhang et al. [24] proposed an energy-efficient trajectory planning method, where one UAV was powered by both a charging station and solar energy. They developed a module-free RL method to optimize trajectory, aiming at maximizing energy efficiency and average data rate. In [25], the UAV energy transfer optimization was

first built as a Markov decision process model. Then, a deep Q-network (DQN) based approach was presented to maximize the UAV's long-term utility. Fu et al. [26] also adopted Q-learning to optimize the UAV's trajectory when collecting sensor data from ground devices, which could minimize energy consumption. Inspired by aerial refueling, Zhu et al. [27] studied the charging UAV and mission UAV without interrupting the mission. They adopted a DDPG with adaptive parameter space noise to maximize the energy charging efficiency. The authors in [28] studied a stochastic energy harvesting system and proposed a modified DQN algorithm to maximize the long-term utility. In [29], a UAV adopted the radio frequency energy transfer to wirelessly charge GSDs. The researchers in [29] presented a DQN based method to decrease the age of information of GSDs. Oubbati et al. [30] employed a multi-agent RL approach to improve transmission and flight energy, aiming at maximizing the total energy received by UAVs. Zhang et al. [31] investigated an MOO problem, with the throughput, energy consumption, and harvested energy taken into account, and proposed an attentional DDPG algorithm to address the problem. For the task offloading and energy harvesting, Seid et al. [32] adopted a model-free multi-agent DDPG to minimize computation cost and resource price in multi-UAV-enabled IoT networks. Fan et al. [33] investigated a UAV trajectory planning problem in multiple WSCs, where the UAV was charged by WSCs. They proposed a multi-head attention mechanism based RL method to shorten the total travel distance of the UAV.

C. Analysis and Motivation

In spite of the considerable efforts devoted to UAV-assisted MEC networks, there exist significant challenges with respect to system modeling and optimization techniques.

System Modeling: Many previous studies adopt one or more UAVs to wirelessly charge GSDs [8], [21], [23], [29]. While it can well support the computing needs of small-scale GSDs, such a scenario is unsuitable for large-scale GSD deployment. This is because UAVs have to charge plentiful GSDs and rapidly consume their battery power, resulting in short-distance flight missions and limited application range. Although GBSs can be deployed to maintain the energy supply of UAVs using the WPT technology [11], [25], they may be unavailable in some extreme scenarios such as military attacks and natural disasters.

To overcome the issue above, some studies [9] concentrate on the integration of HAP into UAV-assisted MEC systems. By establishing effective collaboration between HAP and UAV, it becomes possible to provide energy supply and computing services to GSDs. For instance, an HAP equipped with a laser beam can be deployed in the stratosphere to transmit the laser energy to UAVs. These UAVs harness the harvested laser energy to process computation tasks from GSDs. Moreover, UAVs can play an energy relay role to transmit their harvested energy from HAP to GSDs using WPT technology. Therefore, HAP-and-UAV collaborative MEC is a practical scenario. Nevertheless, there has been very little work on the collaborative scenario. Meanwhile, the integration also brings new challenges to the optimization of UAV trajectories. This is because the amount of energy harvested

TABLE I
COMPREHENSIVE COMPARISON BETWEEN THE RELATED WORKS AND OURS

Reference		[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]
System modeling	Energy source	UAV	UAV	HAP	GBS	GBS	UAV	WSC	GBS	GBS	UAV	UAV	UAV	GBS	GBS
	Single-objective	✓		✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	
	Multi-objective		✓								✓				✓
Optimization technique	Traditional	✓			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	SORL		✓	✓											
	MORL														
Reference		[21]	[22]	[23]	[24]	[25]	[26]	[27]	[28]	[29]	[30]	[31]	[32]	[33]	Ours
System modeling	Energy source	UAV	UAV	UAV	Sun	GBS	WSC	UAV	UAV	UAV	UAV	UAV	UAV	WSC	HAP
	Single-objective	✓	✓	✓		✓	✓	✓	✓	✓	✓		✓	✓	
	Multi-objective				✓	✓						✓	✓		✓
Optimization technique	Traditional	✓	✓	✓											
	SORL				✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	
	MORL														✓

by the UAV from the HAP depends on the distance between them. In general, the amount of harvested energy has a nonlinear relationship with the distance. Further, the distance only relies on the UAV's positions since the HAP maintains a quasi-static position and fixed floating height. Therefore, it is challenging to optimize the trajectories of the UAV to improve its energy efficiency and collect tasks as many as possible.

On the other hand, simultaneously optimizing energy efficiency and the number of collected tasks is one of the research hotspots in UAV-assisted MEC with wireless charging. Many previous studies optimize the two objectives separately, ignoring the conflict and easily causing biased optimization results. Furthermore, the preference between objectives may vary over time. For example, when a UAV operates with low battery levels, end-users may prioritize the reduction of energy consumption. Conversely, when UAVs possess ample energy reserves, they may concentrate on maximizing the number of collected tasks. Nevertheless, all existing works neglect the changing preferences, thus their methods are unsuitable for UAV-assisted MEC scenarios with dynamic preferences.

Optimization Technique: Traditional methods, including iterative algorithm [12], convex optimization [10], [21], Lagrange multiplier [14], heuristics [22], and metaheuristics [23], demonstrate satisfactory performance in addressing diverse optimization problem in static MEC networks, such as a UAV remains stationary at a fixed location throughout the entire flight process. Nevertheless, these methods face challenges in adapting to dynamic environments, especially when UAVs exhibit rapid mobility and tasks arrive in an unpredictable manner. The frequent dynamics and uncertainties often necessitate the repeated execution of these methods, causing increased computational overhead and slow response speed. Therefore, these traditional approaches may not be well-suited for quickly responding to the requirements of users in the context of dynamic UAV-assisted MEC network environments.

Unlike traditional methods, RLs are able to solve complex optimization problems using limited prior information from dynamic MEC environments. This is because RL-based methods can quickly adjust their behaviors to adapt to changing environments. Nevertheless, all the RLs, see [8], [9], [24], [25], [26], [27], [28], [29], [30], [31], [32], are single-objective RLs (SORLs), which first convert multiple objectives into a single

objective via linear scalarization, and then design a corresponding scalar reward and optimize it. Therefore, these SORLs have limitations to the situation in which the preferences assigned to each objective, are predetermined and remain constant throughout the learning process. That is to say, an SORL solely learns the optimal policy over a given preference after a run. In most cases, however, the preferences between objectives cannot be predetermined and may vary over time, especially when the UAV's battery level decreases or increases rapidly. For example, when a UAV operates with low battery levels, end-users may prioritize the reduction of energy consumption. On the contrary, when UAVs possess ample energy reserves, they may concentrate on maximizing the number of collected tasks. Therefore, we have to re-execute SORL algorithms over new preferences, which may be time-consuming. As an emerging MOO algorithm, MORLs with dynamic preference settings can adapt to varying preferences by learning weight-dependent multi-objective Q-value vectors [34]. This allows for more efficient adaptation to changing preferences without the need for extensive re-training. This motivates us to adapt MORL to the ETWC problem.

Table I shows a comprehensive comparison between the related works and ours in terms of system modeling and optimization technique. It can be seen that the energy source comes from the HAP in our UAV-assisted MEC system. Moreover, we adopt MORL to handle the modeled multi-objective optimization problem.

III. SYSTEM MODEL AND PROBLEM FORMULATION

As illustrated in Fig. 1, we study a collaborative MEC system including an HAP and a UAV, which cooperate with each other to provide energy supply and computing services to a set of GSDs, denoted as $\mathcal{M} = \{1, \dots, M\}$. These GSDs are randomly distributed within a rectangular region and they can generate computation tasks over time. It is assumed that ground-based cellular communication infrastructures are unavailable for GSDs, such as remote areas or disaster areas. This paper adopts the rotary-wing UAV to provide the energy supply and computing service for GSDs because this kind of UAV can maintain close proximity to the GSDs at a low height. The HAP floats in the stratosphere in a quasi-static manner and its floating height is fixed. The UAV can transmit its energy to GSDs, enabling

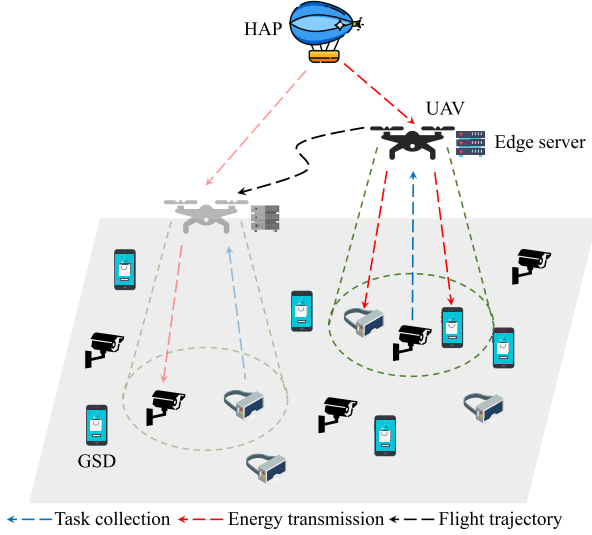


Fig. 1. UAV-assisted MEC system.

them to sustainably implement specific tasks, such as environmental monitoring. Equipped with a high-power laser beam, the HAP can wirelessly charge the UAV, thus maintaining its flight missions. A discrete-time system is considered in this paper and let τ denote the time duration in each time slot. Assume the whole flight mission (i.e., a task collection and energy transmission period) has T time slots, denoted by set $\mathcal{T} = \{1, \dots, T\}$. We suppose that the UAV flies at a constant speed. Table II summarizes the main notations used in system model and reinforcement learning.

A. Task Model

In general, a computation task can be represented as a two-tuple $\langle \mathcal{I}, \mathcal{C} \rangle$, where $\mathcal{I}$ and $\mathcal{C}$ indicate the input data size and computation intensity of the task, respectively. Note that $\mathcal{C}$ can reveal how many CPU cycles are required to handle one bit input data. Similar to [2], [35], the arrival of computation tasks in GSD is modeled as a sequence of independent identically distributed Bernoulli stochastic processes. Let $\xi^m \in [0, 1]$ denote the parameter of the Bernoulli stochastic process corresponding to GSD m . The parameters for different GSDs may be different because of their heterogeneity. For example, a video monitoring device has a quicker data generation rate than a temperature monitoring device. Let I_t^m represent whether a computation task is generated by GSD m in time slot t . If the GSD generates a computation task in t , $I_t^m = 1$, and $I_t^m = 0$, otherwise. A generated computation task by a GSD can be saved in its task queue. We denote by Υ_t^m the number of computation tasks in the queue of GSD m in t , and it is updated by

$$\Upsilon_{t+1}^m = \min\{\Upsilon_t^m + I_t^m, \Upsilon_{\max}^m\}, \quad \forall t \in \mathcal{T}, m \in \mathcal{M}, \quad (1)$$

where $\Upsilon_{\max}^m$ represents the task queue capacity that limits the maximum number of tasks that GSD m can store in its task queue. We suppose the task queue capacity is the same for all GSDs. It is worth noting that if the task queue reaches its maximum capacity, newly arrived tasks may overwrite older tasks or

TABLE II
MAIN NOTATIONS USED IN SYSTEM MODEL AND REINFORCEMENT LEARNING

Notation	Definition
Notation used in system model	
$\mathcal{A}$	Collection mirror area
$\mathcal{C}$	Computation intensity of a task
$\mathcal{C}^U$	Energy conversion efficiency of the UAV
D	Initial laser beam size
$E_{\max}^U$	Maximum battery capacity of the UAV
E_{total}^U	Total energy efficiency of the UAV
f^U	Computing capability of the UAV
g_0	Channel power gain
$\mathcal{I}$	Input data size of a task
m	The m -th GSD
M	Number of GSDs
$\mathcal{M}$	Set of GSDs
$N_{\max}$	Maximum task number the UAV can store
N_{total}	Total number of collected tasks
P^H	Laser transmission power of the HAP
P^U	Energy transmission power of the UAV
$R_{\max}$	Maximal horizontal coverage of the UAV
t	The t -th time slot
T	Number of time slots
$\mathcal{T}$	Set of time slots
$\mathcal{X}$	Combined optical efficiency
η	Attenuation coefficient
ϑ	Angular spread of laser beam
$\theta_{\max}$	Maximal azimuth angle
ξ^m	Parameter of Bernoulli stochastic process for GSD m
ρ_t^m	Task upload priority of GSD m in time slot t
Notation used in reinforcement learning	
a	Action
$\mathcal{A}$	Action space
$\mathcal{B}$	Experience replay buffer
n	Number of objectives
$N_{\max}^{\text{eps}}$	Maximum number of episodes
$\mathbf{Q}_{\pi}$	Multi-objective Q-value vector
$\mathbf{r}_t$	Vectorial reward at time step t
s	State
$\mathcal{S}$	State space
γ	Discount factor
ψ	Balance weight
Ω	Preference space

be dropped altogether. Consequently, ensuring the timely upload of computation tasks from GSDs to the UAV becomes crucial. Since variations in the number of tasks stored and task generation rates across different devices, their priorities for uploading tasks differ among them. We define the task upload priority ρ_t^m of GSD m as

$$\rho_t^m = \xi^m \frac{\Upsilon_t^m}{\Upsilon_{\max}^m}, \quad \forall t \in \mathcal{T}, m \in \mathcal{M}. \quad (2)$$

The task upload priority of a GSD is influenced not only by the ratio of the number of tasks generated by the GSD to the queue capacity but also by the task generation rate.

B. Energy Harvesting Model and Computing Model

Similar to previous studies [2], [8], the Cartesian coordinate system is adopted to model the locations of the HAP, UAV, and GSDs. We assume that the locations of GSDs are static. Let $\mathbf{c}^m = (x^m, y^m, 0)$ be the coordinate of GSD $m \in \mathcal{M}$. The UAV cruises with planned trajectories at a constant speed and its flight height U is fixed, where U is a positive constant. Let $\mathbf{c}_t^U = (x_t^U, y_t^U, U)$ indicate the coordinate of the UAV in time slot t

and it may change over time. The HAP can be aware of the UAV's movement and adjust its laser beam accordingly. Let $\mathbf{c}^H = (x^H, y^H, H)$ denote the coordinate of the HAP, where H is the floating height.

Let P^H indicate the HAP's laser transmission power. The HAP employs the fixed transmission power to wirelessly charge the UAV. According to [9], [11], the energy harvested by the UAV from the HAP can be obtained by

$$E_t^H = C^U \tau P^H \frac{A \mathcal{X} e^{-\eta d_t^{\text{HU}}}}{(D + \vartheta d_t^{\text{HU}})^2}, \quad \forall t \in \mathcal{T}, \quad (3)$$

where $C^U \in (0, 1)$ denotes the UAV's energy conversion efficiency, τ denotes the time duration in each time slot, A is the collection mirror area of laser beam, $\mathcal{X}$ is the combined optical efficiency, η is the attenuation coefficient, D is the initial laser beam size, and ϑ is the angular spread of laser beam. $d_t^{\text{HU}} = \|\mathbf{c}^H - \mathbf{c}^U\|_2$ is the distance between the HAP and UAV, where $\|\cdot\|_2$ denotes the euclidean 2-norm. According to (3), E_t^H mainly relies on τ and d_t^{HU} under other parameter values are kept constant.

The UAV harvests laser energy from the HAP and transmits the harvested energy to GSDs while it collects computation tasks from a specific GSD. Note that the ranges of the energy transmission and task collection are limited because the UAV owns limited communication coverage. Thus, the UAV can only transmit energy and collect tasks within its communication coverage. This depends on its maximal azimuth angle $\theta_{\max}$ and fixed flight height U . Let $\mathcal{M}_t^C$ denote the set of GSDs covered, which is obtained by

$$\mathcal{M}_t^C = \{m | d_t^m \leq R_{\max}\}, \quad \forall t \in \mathcal{T}, m \in \mathcal{M}, \quad (4)$$

where $d_t^m = \sqrt{(x_t^U - x^m)^2 + (y_t^U - y^m)^2}$ denotes the horizontal distance between the UAV and GSD m . $R_{\max} = U \cdot \tan(\theta_{\max})$ is the maximal horizontal coverage of the UAV. The UAV selects a GSD as the specific device for task collection. In time slot t , the specific GSD, denoted by $\tilde{m}_t$, has largest task upload priority, i.e., $\tilde{m}_t = \arg \max_{m \in \mathcal{M}_t^C} \rho_t^m$, where ρ_t^m is the task upload priority of GSD m in t .

We consider the practical air-to-ground channel model, which combines line-of-sight (LoS) and non-line-of-sight (NLoS) links [8]. LoS signifies the propagation of signals in a direct trajectory from transmitters to receivers, without substantial impediments or reflective occurrences. Conversely, NLoS denotes the necessity for signals to reach receivers through reflections or scattering, often involving obstacles or multiple reflections. The path loss between the UAV and GSD m in t with respect to LoS and NLoS is defined as

$$\Gamma_t^m = \begin{cases} g_0 (d_t^m)^{-\alpha}, & \text{LoS link} \\ \eta^{\text{NLoS}} g_0 (d_t^m)^{-\alpha}, & \text{NLoS link} \end{cases} \quad (5)$$

where g_0 is the channel power gain for 1 m reference distance, α represents the path loss exponent, and η^{NLoS} indicates the additional attenuation coefficients of NLoS link. The LoS probability of GSD m in t is calculated as

$$PL_t^m = \frac{1}{1 + c_1 \cdot e^{c_2 \theta_t^m - c_1}}, \quad (6)$$

where c_1 and c_2 are two constants that rely on the carrier frequency and environmental conditions. $\theta_t^m = \frac{180}{\pi} \sin^{-1}(U/d_t^m)$ is the elevation angle of the UAV and GSD m in degree. PL_t^m is influenced by the environmental conditions and distance between the UAV and GSD m . Suppose that the uplink and downlink channels exhibit approximate equality. Thus, the channel power gain between the UAV and GSD m in t can be defined as

$$g_t^m = (PL_t^m + \eta^{\text{NLoS}} PN_t^m) g_0 (d_t^m)^{-\alpha}, \quad (7)$$

where $PN_t^m = 1 - PL_t^m$ is the NLoS probability. Based on (7), the signal-to-noise ratio (SNR) between the UAV and GSD m in t is defined as

$$\Psi_t^m = \frac{P^G \cdot g_t^m}{\sigma^2}, \quad (8)$$

where P^G and σ^2 are the uploading power of GSD and noise power, respectively. The communication outage may arise because there are obstacles at a low altitude. Therefore, Ψ_t^m is not less than the required threshold SNR Ψ_{th} for establishing successful wireless connectivity. At the end of t , the UAV can finish its task collection process. Consider the communication outage, the number of collected tasks, N_t^C , is obtained by

$$N_t^C = \begin{cases} \Upsilon_t^{\tilde{m}_t}, & \text{if } \Psi_t^{\tilde{m}_t} \geq \Psi_{\text{th}} \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

Let P^U indicate the energy transmission power of the UAV for transmitting radio frequency signals to GSDs. The energy harvested at GSD $m \in \mathcal{M}_t^C \setminus \{\tilde{m}_t\}$ from the UAV in t can be expressed as [9]

$$E_t^m = C^G \tau P^U g_t^m, \quad \forall t \in \mathcal{T}, m \in \mathcal{M}_t^C \setminus \{\tilde{m}_t\}, \quad (10)$$

where C^G is the energy conversion efficiency of GSD. The total energy for the UAV transmitting energy to GSDs in t is obtained by

$$E_t^{\text{Tr}} = \sum_{m \in \mathcal{M}_t^C \setminus \{\tilde{m}_t\}} E_t^m, \quad \forall t \in \mathcal{T}. \quad (11)$$

We consider that the UAV keeps a computing queue in order to store the collected tasks, which are awaiting for further handling. It should note that task collection delay and associated energy consumption can be neglected because the UAV can sufficiently close GSDs. The UAV can only store the specified number of computation tasks due to its limited storage capacity, denoted by $N_{\max}$. In each time slot, the UAV has to execute the unfinished tasks stored in the computing queue. Let N_t^U denote the number of unfinished tasks in the queue, and its value is an integer limited between 0 and $N_{\max}$. Within a time slot, the UAV can only process a fixed number of computation tasks because of its limited computing capability, denoted by f^U . Thus, the number of tasks executed by the UAV within a time slot can be obtained by $\min\{N_t^U, \varphi\}$, where $\varphi = \lfloor \tau f^U / \mathcal{I} \rfloor$ is maximum number of tasks the UAV can process in each time slot. Based on N_t^U and φ , the computing queue is updated, and the number of queuing tasks, N_t^Q , can be calculated by

$$N_t^Q = \max\{N_t^U - \varphi, 0\}, \quad \forall t \in \mathcal{T}. \quad (12)$$

At the end of t , the UAV can finish its task collection procedure. By employing (9) and (12), we update the number of unfinished tasks in $t + 1$, N_{t+1}^U , expressed as

$$N_{t+1}^U = \min\{N_t^C + N_t^Q, N_{\max}\}, \quad \forall t \in \mathcal{T}. \quad (13)$$

The energy consumption of the UAV for executing $\min\{N_t^U, \varphi\}$ computation tasks is calculated as

$$E_t^P = \kappa \cdot \min\{N_t^U, \varphi\} \mathcal{IC} \cdot (f^U)^2, \quad \forall t \in \mathcal{T}, \quad (14)$$

where κ denotes the effective capacitance coefficient.

C. Problem Formulation

In each time slot, the UAV harvests the laser energy from the HAP and transmits the harvested energy to GSDs (except the specific device) within its coverage. Meanwhile, the UAV is in charge of gathering computation tasks from the specific GSD and processing them. Hence, the UAV's energy consumption encompasses both the energy consumed for transmitting to GSDs and energy consumed for processing collected tasks. Let E_t^{Tr} and E_t^P denote the transmitting energy consumption and processing energy consumption, respectively. Let E_t^U denote the energy level of the UAV at the beginning of t , and it is updated by

$$E_{t+1}^U = \min\{E_{\max}^U, \max\{0, E_t^U + E_t^H - E_t^{\text{Tr}} - E_t^P\}\}, \quad \forall t \in \mathcal{T}, \quad (15)$$

where $E_{\max}^U$ is the maximum battery capacity of the UAV. The total energy efficiency of the UAV is defined as

$$E_{\text{total}} = \sum_{t=1}^T E_t^U. \quad (16)$$

On the basis of (9), the total number of computation tasks collected by the UAV during time duration T can be calculated by

$$N_{\text{total}} = \sum_{t=1}^T N_t^C. \quad (17)$$

Let $\mathbf{C} = \{\mathbf{c}_t^U | \forall t \in \mathcal{T}\}$ represent the set of the UAV's coordinates during T time slots. We formulate the ETWC problem as an MOO problem, with E_{total} and N_{total} maximized simultaneously, via optimizing the UAV's flight trajectories in each time slot, as follows.

$$\max_{\mathbf{C}} (E_{\text{total}}, N_{\text{total}}) \quad (18)$$

subject to:

$$\text{C1} : 0 \leq x_t^U \leq x_{\max}, \quad \forall t \in \mathcal{T},$$

$$\text{C2} : 0 \leq y_t^U \leq y_{\max}, \quad \forall t \in \mathcal{T},$$

$$\text{C3} : E_t^U \leq E_{\max}^U, \quad \forall t \in \mathcal{T},$$

$$\text{C4} : d_t^m \leq R_{\max}, \quad \forall m \in \mathcal{M}_t^C, t \in \mathcal{T}.$$

C1 and C2 together show the UAV's flight region. C3 restricts that the UAV's energy level is not exceeding its maximum battery capacity. C4 denotes that the UAV can only transmit

the harvested energy to GSDs and collects computation tasks from them within its communication coverage. One can understand that to maximize N_{total} , the UAV should follow a suitable trajectory during its flight mission, so as to it can seek the specific GSD and gather its tasks. However, as the number of collected tasks increases, the UAV's energy efficiency decreases correspondingly. This is because the UAV needs to handle more computation tasks collected by it. Thus, it is easily understood that two objectives, i.e., maximization of E_{total} and maximization of N_{total} , are inherently conflicting with each other.

IV. OVERVIEW OF MOMDP

An MOMDP [34] is normally represented by a five-tuple $\langle \mathcal{S}, \mathcal{A}, \mathbf{r}, \Omega, f \rangle$, where $\mathcal{S}$ is the state space consisting of possible states in which the environment can be. $\mathcal{A}$ is the action space that is the set of actions the agent can take. $\mathbf{r}(s, a)$ is the vectorial reward function used to evaluate the performance of action a taken by the agent in state s . Ω is the preference space adopted to simulate various preferences between objectives. $f_{\mathbf{w}}(\mathbf{r}(s, a))$ is the scalarization function that can generate a scalar user utility using preference $\mathbf{w} \in \Omega$. This paper focuses on the linear scalarization function, which aggregates $\mathbf{r}(s, a)$ into a scalar reward using weighted sum, i.e., $f_{\mathbf{w}}(\mathbf{r}(s, a)) = \mathbf{w} \cdot \mathbf{r}(s, a)$. It is seen that if $\mathbf{w}$ is held constant at a specific value, the MOMDP simplifies into a single-objective MDP.

A policy π is a state-to-action mapping, i.e., $\pi : \mathcal{S} \rightarrow \mathcal{A}$. Through interacting with the environment, an agent gathers information and acquires knowledge to learn an optimal policy that maximizes the expected cumulative reward. The action-value function is the vector of the expected return, which can be described as

$$\begin{aligned} \mathbf{Q}_{\pi}(s, a) &= \mathbb{E}_{\pi} [\mathbf{R}_t | s_t = s, a_t = a] \\ &= \mathbb{E}_{\pi} \left[\sum_{k=t}^T \gamma^{k-t} \cdot \mathbf{r}_k \middle| s_t = s, a_t = a \right], \end{aligned} \quad (19)$$

where $\mathbf{R}_t$ represents the cumulative discounted reward at time step t ; $\gamma \in [0, 1]$ is the discount factor; $\mathbf{r}_k = (r_k^1, \dots, r_k^n)$ indicates the immediate vectorial reward received at time step k , where n is the number of objectives. Since each element in $\mathbf{r}_k$ is associated with an individual objective, $\mathbf{Q}_{\pi}(s, a)$ is a multi-objective Q-value (MOQ) vector with n elements. For the ETWC problem, we have $n = 2$, namely, r_k^1 and r_k^2 are associated with E_{total} and N_{total} , respectively.

V. THE PROPOSED ALGORITHM

A. MOMDP Model

To tackle the ETWC problem using an MORL algorithm, it is imperative to establish a dedicated MOMDP model tailored to the problem at hand. This involves systematically defining the state space, action space, and reward function.

1) *State Space*: A state space is the set of possible states in which the UAV-assisted MEC environment can be. It is assumed that the UAV can only observe its own coordinate position $\mathbf{c}_t^U$, the number of unfinished tasks N_t^U , the number of collected

tasks N_t^C , and energy level E_t^U , in time slot t . To be specific, the state space is defined as

$$\mathcal{S} = \{s_t | s_t = (c_t^U, N_t^U, N_t^C, E_t^U), \forall t \in \mathcal{T}\}, \quad (20)$$

where s_t is the MEC system environment state in t .

2) *Action Space*: We assume that the UAV can select one of four directions to move at its current location in each time slot. Thus, the action space can be defined as

$$\mathcal{A} = \{a_t | a_t \in \{N, S, E, W\}, \forall t \in \mathcal{T}\}, \quad (21)$$

where a_t is an action the UAV takes in state s_t . Actions N, S, E , and W represent that the UAV moves to the north, south, east, and west from the current location, respectively.

3) *Reward Function*: The ETWC problem aims to maximize E_{total} and N_{total} simultaneously. As a result, after the agent executes action a_t , we can define the vectorial reward received by it as

$$\mathbf{r}_t = (r_t^E, r_t^N) = \begin{cases} \left(\frac{E_t^U}{10}, N_t^C \right), & \text{if } \mathbb{1}_t = 0 \\ \left(\delta_1 \frac{E_t^U}{10}, \delta_2 N_t^C \right), & \text{if } \mathbb{1}_t = 1 \end{cases} \quad (22)$$

where r_t^E and r_t^N denote the two scalar rewards that are associated with E_t^U and N_t^C , respectively. Coefficient $\frac{1}{10}$ ensures that r_t^E and r_t^N have the same order of magnitude. This can enable the simultaneous optimization of two objectives without introducing any biases, thus achieving a good balance between them. An indicator variable, $\mathbb{1}_t$ can be one if the UAV moves outside of the restricted region in t ; otherwise, $\mathbb{1}_t = 0$. Based on the variable, one can punish the two scalar rewards, $\frac{E_t^U}{10}$ and N_t^C when $\mathbb{1}_t = 1$. As a result, the penalty coefficients δ_1 and δ_2 are adopted to decrease $\frac{E_t^U}{10}$ and N_t^C , respectively.

Lemma 1: The maximization of the expected vectorial return $\mathbb{E}[\mathbf{R}_1]$ is equal to the simultaneous maximization of E_{total} and N_{total} .

Proof 1: Based on the reward $\mathbf{r}_t = (r_t^E, r_t^N)$ defined in (22), we can obtain the vector of cumulative discount rewards received by the agent during T time slots. It is also known as the vectorial return, $\mathbf{R}_1 = (R_1^E, R_1^N)$, which is calculated as

$$R_1^E = \sum_{t=1}^T \gamma^{t-1} r_t^E = \sum_{t=1}^T \gamma^{t-1} (1 - \mathbb{1}_t + \delta_1 \mathbb{1}_t) \frac{E_t^U}{10}, \quad (23)$$

$$R_1^N = \sum_{t=1}^T \gamma^{t-1} r_t^N = \sum_{t=1}^T \gamma^{t-1} (1 - \mathbb{1}_t + \delta_2 \mathbb{1}_t) N_t^C. \quad (24)$$

According to (16), (17), (23), and (24), the maximization of the expected vectorial return $\mathbb{E}[\mathbf{R}_1]$ is equal to the simultaneous maximization of E_{total} and N_{total} .

B. MORL-TER Algorithm

MORLs have attracted increasingly more research attention [34], [36], [37], [38], [39]. Unlike SORL with a scalar reward, MORL has a vectorial reward in which each element corresponds to a specific objective. Through learning weight-dependent MOQ vectors, envelope multi-objective Q-learning (EMOQL) has great potential to deal with MEC scenarios with

Algorithm 1: MORL-TER for the ETWC Problem.

Input: maximum number of episodes $N_{\text{max}}^{\text{eps}}$, discount factor γ , balance weight ψ , greedy exploration ε .

- 1: Initialize the preference space Ω by the systematic method;
- 2: Initialize policy MOQ network $\mathbf{Q}(s, a, \mathbf{w}; \theta)$ with parameters θ ;
- 3: Initialize target MOQ network $\bar{\mathbf{Q}}(s, a, \mathbf{w}; \bar{\theta})$ with $\bar{\theta} = \theta$;
- 4: Set the experience replay buffer $\mathcal{B} = \emptyset$;
- 5: **for** $k = 1, \dots, N_{\text{max}}^{\text{eps}}$ **do**
- 6: Sample a preference $\mathbf{w}_k$ from Ω ;
- 7: Set the trace $\Gamma_k = \emptyset$;
- 8: **for** $t = 1, \dots, T$ **do**
- 9: Observe current state s_t ;
- 10: Choose action a_t using (25);
- 11: Take action a_t and move the UAV to the next location;
- 12: Receive reward $\mathbf{r}_t$ and observe next state s_{t+1} ;
- 13: Store transition $(s_t, a_t, \mathbf{r}_t, s_{t+1})$ in Γ_k ;
- 14: **if** $\mathcal{B}$ is full **then**
- 15: Sample N_{tra} transitions from $\mathcal{B}$, denoted by $\mathcal{D}$;
- 16: Sample N_{w} preferences from Ω , denoted by $\mathcal{W}$;
- 17: **for** each transition $(s_j, a_j, \mathbf{r}_j, s_{j+1}) \in \mathcal{D}$ **do**
- 18: Calculate $\text{Loss1}(\theta)$ and $\text{Loss2}(\theta)$ using (26) and (28), respectively;
- 19: Perform gradient descent step on (29) to update the policy MOQ network;
- 20: **end for**
- 21: **end if**
- 22: Every $\bar{N}$ time steps, synchronize target MOQ network, i.e., $\bar{\theta} = \theta$;
- 23: Update ψ and ε ;
- 24: **end for**
- 25: Update $\mathcal{B}$ based on Γ_k by $\text{TER}(\mathcal{B}, \Gamma_k, \gamma)$;
- 26: **end for**

Output: the policy MOQ network $\mathbf{Q}(s, a, \mathbf{w}; \theta)$.

dynamic preferences [34]. This algorithm has demonstrated successful applications across diverse domains, including machine control, task offloading [39], and dialog systems [34]. This motivates us to adapt the EMOQL algorithm to the ETWC problem concerned in the paper.

The framework of the proposed MORL-TER, as depicted in Fig. 2, comprises two components: the agent and MEC system environment. The agent can be typically deployed on the UAV and engages in interactive exchanges with the environment. It encompasses both the execution and training stages. In the agent, there are two different MOQ networks (i.e., policy and target MOQ networks), and their architectures are the same. The policy MOQ network is used to choose the actions. The target MOQ network is adopted to avoid overestimation of MOQ values. Note that the policy MOQ network in the execution stage is exactly the same as the one in the training phase for the purpose of presentation. In the execution stage, we use the policy MOQ network to select actions, i.e., the UAV's flight directions. In the training stage, the multiple transitions are randomly sampled to train the policy MOQ network.

Throughout the execution stage, the agent continually interacts with the environment, enabling it to observe the MEC

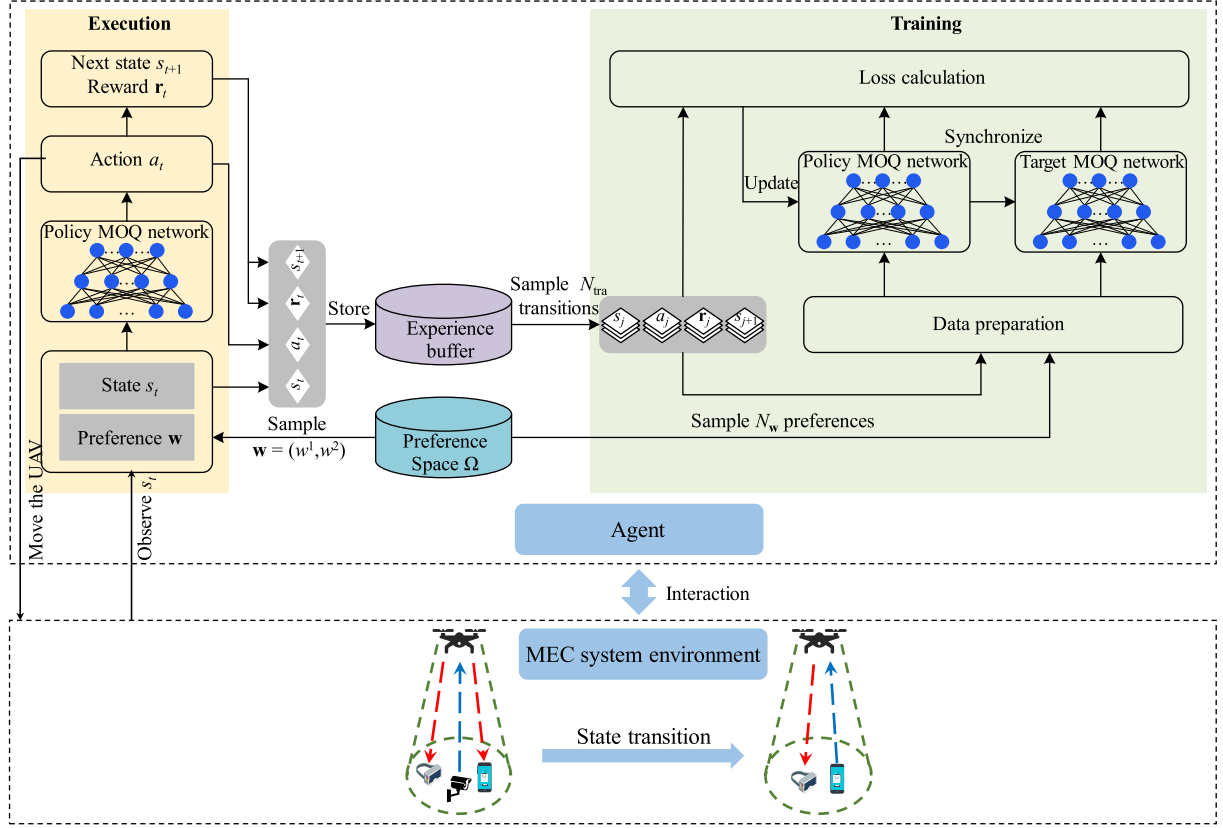


Fig. 2. Framework of MORL-TER.

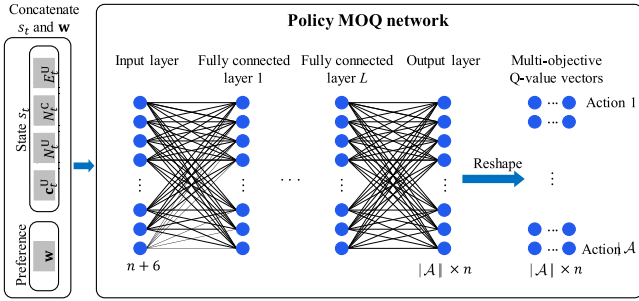


Fig. 3. Network architecture.

system's states and subsequently make decisions to the movement of the UAV. The architecture of the policy MOQ network is illustrated in Fig. 3, which is a fully connected neural network with L layers. The policy MOQ network first concatenates the observed state $s_t \in \mathcal{S}$ and sampled preference $\mathbf{w} \in \Omega$. Before inputting the network, the concatenated results should be normalized because of different dimensionality. Then, the normalized outcomes are fed into the policy MOQ network and it outputs $|\mathcal{A}| \times n$ Q-values. Finally, these Q-values are reshaped into a $|\mathcal{A}| \times n$ matrix in which each row is associated with a multi-objective Q-value vector with n components. To adopt conventional action choice methods, for instance, ε -greedy exploration, an action $a_t \in \mathcal{A}$ is chosen based on left multiplying the matrix by $\mathbf{w}$. The agent executes a_t to transition the UAV from its

current location to the next location based on the observation of the next state s_{t+1} . Meanwhile, it obtains a vectorial reward $\mathbf{r}_t$ from the environment. In the training stage, a mini-batch of transitions and multiple preferences are first sampled from the experience buffer and preference space, respectively. Then, the agent adopts these sampled transitions and preferences to train its policy MOQ network, thereby generalizing various preferences.

The pseudocode of MORL-TER is illustrated in Algorithm 1, which is based on EMOQL [34], with a trace-based experience replay (TER) scheme incorporated. Similar to [40], we adopt the systematic method to produce multiple evenly distributed weight vectors (i.e., preference space), so as to mimic diverse preferences. The method has two important parameters n and σ , where n and σ are the numbers of objectives and divisions. Based on the two parameters, the systematic method can generate $\Phi = C_{n+\sigma-1}^{n-1}$ uniformly distributed weight vectors, denoted by $\Omega = \{\mathbf{w}_1, \dots, \mathbf{w}_\Phi\}$. For instance, we have $n = 2$ for the ETWC problem with two objectives. If $\sigma = 149$, the systematic method generates $\Phi = 150$ weight vectors to represent various preferences.

We initialize the policy MOQ network using the random parameters θ (step 2). The parameters of the target MOQ network are initially set to $\bar{\theta}$ (step 3). Once the policy MOQ network is initialized, the agent commences interaction with the environment so that it can observe current environment state s_t (step 9). At the beginning of each episode $k = 1, \dots, N_{\max}^{\text{eps}}$, a current

preference $\mathbf{w}_k$ is randomly sampled from Ω , where $N_{\max}^{\text{eps}}$ is the maximum number of episodes. The action a_t is made based on (25), i.e., the ε -greedy exploration.

$$a_t = \begin{cases} \text{a random action from } \mathcal{A}, & \text{with probability } \varepsilon \\ \text{argmax}_{a \in \mathcal{A}} \mathbf{w}_k \mathbf{Q}(s_t, a, \mathbf{w}_k; \theta), & \text{with probability } 1 - \varepsilon \end{cases} \quad (25)$$

where $\varepsilon \in [0, 1]$ denotes the probability of random exploration and updated over time steps. The agent takes a_t and moves the UAV from current location to next location (step 11). Afterward, it obtains a vectorial reward $\mathbf{r}_t$ from the environment and observe next state s_{t+1} (step 12). In step 13, the transition $(s_t, a_t, \mathbf{r}_t, s_{t+1})$ is stored in the current trace Γ_k .

If the buffer $\mathcal{B}$ is full, N_{tra} transitions, denoted by $\mathcal{D}$, are randomly sampled from $\mathcal{B}$ to train the policy MOQ network (step 15). Meanwhile, $N_{\mathbf{w}}$ preferences, denoted by $\mathcal{W}$, are randomly sampled from the preference space Ω (step 16). For each sampled transition $(s_j, a_j, \mathbf{r}_j, s_{j+1}) \in \mathcal{D}$, we first calculate the loss function $\text{Loss1}(\theta)$, defined as

$$\text{Loss1}(\theta) = \mathbb{E} [\|\mathbf{y}_j - \mathbf{Q}(s_j, a_j, \mathbf{w}_i; \theta)\|_2^2], \mathbf{w}_i \in \mathcal{W}, \quad (26)$$

where $\mathbf{y}_j = \mathbf{r}_j$ if s_{j+1} is the terminal state; otherwise, $\mathbf{y}_j$ is obtained by

$$\mathbf{y}_j = \mathbf{r}_j + \gamma \bar{\mathbf{Q}}\left(s_{j+1}, \text{argmax}_{a \in \mathcal{A}, \mathbf{w}' \in \mathcal{W}} \mathbf{w}_i \mathbf{Q}(s_{j+1}, a, \mathbf{w}'; \theta), \mathbf{w}_i; \bar{\theta}\right). \quad (27)$$

According to (27), we adopt double Q-learning to train the policy MOQ network, so as to avoid the overestimation of multi-objective Q-values. In addition, preserving the envelope $\text{argmax}_{a \in \mathcal{A}, \mathbf{w}' \in \mathcal{W}} \mathbf{w}_i \mathbf{Q}(s_{j+1}, a, \mathbf{w}'; \theta)$ in our method facilitates the rapid alignment of a preference $\mathbf{w}'$ with optimal policy and transitions, even those explored under different preferences. This means the sampled transitions can be well adapted to optimize the preference $\mathbf{w}'$, finding the optimal policy corresponding to $\mathbf{w}'$. In contrast, scalarized updates optimizing scalar utility lack the ability to leverage $\max_a \mathbf{Q}(s_j, a, \mathbf{w}_i; \theta)$ information for updating optimal policies aligned with different preferences. Therefore, the envelope updates can have better sample efficiency.

However, in practice, directly optimizing the loss function $\text{Loss1}(\theta)$ poses significant challenges because there are a large number of discrete solutions in the optimal frontier. This makes the function's landscape become considerably non-smooth. Thus, an auxiliary loss function, $\text{Loss2}(\theta)$, is used to solve the issue, and it is defined as

$$\text{Loss2}(\theta) = \mathbb{E} [\|\mathbf{w}_i \mathbf{y}_j - \mathbf{w}_i \mathbf{Q}(s_j, a_j, \mathbf{w}_i; \theta)\|], \mathbf{w}_i \in \mathcal{W}. \quad (28)$$

Based on $\text{Loss1}(\theta)$ and $\text{Loss2}(\theta)$, the final loss function $\text{Loss}(\theta)$ can be obtained by

$$\text{Loss}(\theta) = \psi \cdot \text{Loss1}(\theta) + (1 - \psi) \cdot \text{Loss2}(\theta), \quad (29)$$

where $\psi \in (0, 1)$ is a weight coefficient adopted to obtain a balance between $\text{Loss1}(\theta)$ and $\text{Loss2}(\theta)$. We can tardily decrease the value of ψ from 1 to 0, to convert $\text{Loss}(\theta)$ from $\text{Loss1}(\theta)$ to $\text{Loss2}(\theta)$. The loss $\text{Loss1}(\theta)$ initially focuses on narrowing the gap between the predicted Q-values and actual expected rewards.

Algorithm 2: Trace-Based Experience Replay (TER).

Input: experience replay buffer $\mathcal{B}$, trace Γ , discount factor γ .
1: Calculate vectorial return of Γ , $\mathbf{R}_t = \sum_{t=1}^T \gamma^{t-1} \mathbf{r}_t$;
2: **if** $\mathcal{B}$ is not full **then**
3: Add Γ to $\mathcal{B}$;
4: **else**
5: Add Γ to $\mathcal{B}$;
6: Set $l = |\mathcal{B}|$;
7: **for** $i = 1, \dots, l$ **do**
8: Set $\mathcal{B}_i^{\text{dist}} = 0$;
9: **end for**
 // Calculate crowding distance for each trace in $\mathcal{B}$
10: **for** each objective n **do**
11: $\mathcal{B} = \text{sort}(\mathcal{B}, n)$; // sort using each scalar return
12: $\mathcal{B}_1^{\text{dist}} = \mathcal{B}_l^{\text{dist}} = +\infty$;
13: **for** $i = 2, \dots, l - 1$ **do**
14: $\mathcal{B}_i^{\text{dist}} = \mathcal{B}_i^{\text{dist}} + (\mathcal{B}_{i+1}^n - \mathcal{B}_{i-1}^n) / (f_{\max}^n - f_{\min}^n)$;
15: **end for**
16: **end for**
17: Remove the trace with smallest crowding distance from $\mathcal{B}$;
18: **end if**
Output: experience replay buffer $\mathcal{B}$.

On the other hand, the loss $\text{Loss2}(\theta)$ serves as an auxiliary force, guiding the optimization toward the direction of better utility.

The original EMOQL algorithm [34] adopts the standard first-in first-out experience replay buffer, which treats transitions as atomic units when considering them for addition in or deletion from the buffer. However, a big challenge to using the standard experience replay is that the obtained experiences by a preference's optimal policy can be harmful to another preference's training process. In other words, if a policy $\pi_{\mathbf{w}}$ is exclusively trained on experiences obtained via another policy $\pi_{\mathbf{w}'}$, $\pi_{\mathbf{w}}$ will typically diverge from the optimal policy for preference $\mathbf{w}$.

To overcome the challenge above and enhance the optimization performance of the original EMOQL, we develop a trace-based experience replay (TER) scheme, which treats an episode's transitions as one trace. The TER scheme replaces standard transition-based replay with trace-based memorization. Instead of considering each transition independently, the scheme handles traces as atomic units. To grasp the underlying reasons, we examine a trace of experiences that spans from an initial state to a terminal state. In cases where there is a gap in experiences between the initial and terminal states, propagating Q-values from the terminal back to the initial state becomes challenging. This challenge arises because the learning agent must make inferences to bridge the missing link. The standard replay buffers, in their sequential addition and removal of experiences, typically do not encounter this issue. Consequently, for the majority of experiences stored in standard replay buffers, both the preceding and subsequent transitions are readily available. To avoid partial traces, the TER scheme handles traces as atomic units when considering them for addition in or deletion from the experience buffer.

The procedure of the TER scheme is shown in Algorithm 2. We first calculate the vectorial return of the trace Γ , $\mathbf{R}_1 = \sum_{t=1}^T \gamma^{t-1} \mathbf{r}_t$. If the experience buffer $\mathcal{B}$ is not full, the trace

Γ is directly added to $\mathcal{B}$ (steps 2–4). Similar to [36], we also use the crowding distance to reflect the relative diversity of a trace in $\mathcal{B}$. The higher the diversity, the more important the trace. This is because those traces with high diversity, obtained by $\pi_{\mathbf{w}'}$, are also used to train another policy $\pi_{\mathbf{w}}$, which can better maintain the learned optimal policy for preference $\mathbf{w}$. The following describes how to update the experience buffer $\mathcal{B}$ based on the crowding distance of each trace in $\mathcal{B}$.

When the buffer is full, the trace Γ is first added to $\mathcal{B}$. Then, we obtain the crowding distance of each trace in $\mathcal{B}$ (steps 10–16), where $\mathcal{B}_i^{\text{dist}}$ indicates the crowding distance of the i -th trace. $\mathcal{B}_i^n$ represents the n -th scalar return of the i -th trace in $\mathcal{B}$. Parameters $f_{\max}^n$ and $f_{\min}^n$ are the maximal and minimal values of the n -th scalar return. After that, we remove the trace with the smallest crowding distance from buffer $\mathcal{B}$, which guarantees the diversity of the experience replay buffer (step 17). Thus, the TER scheme can improve sample efficiency and reduce replay buffer bias.

C. Complexity Analysis

We analyze MORL-TER's time complexity in terms of the training and testing stages. Compared with the training complexity of the policy MOQ network, the TER scheme's time complexity is trivial and can be neglected. Thus, in the training stage, MORL-TER's time complexity primarily relies on the training complexity of the policy MOQ network. In the light of Fig. 3, the policy MOQ network is comprised of an input layer, an output layer, and L fully connected layers. The number of neurons in the input layer is $n + 6$, where n is the number of objectives. Let β_i represent the number of neurons in the i -th fully connected layer, $i = 1, \dots, L$. The output layer has $|\mathcal{A}| \times n$ neurons, where $|\mathcal{A}| = 4$ is the number of actions. Note that we have $\beta_0 = n + 6$ and $\beta_{L+1} = |\mathcal{A}| \times n$. MORL-TER's time complexity is $O(N_{\text{eps}}^{\max} \times T \times (\sum_{i=1}^{L+1} \beta_{i-1} \times \beta_i) \times N_{\text{tra}})$ in the training stage, where $N_{\text{eps}}^{\max}$ is the maximum number of episodes, T is the number of time slots, and N_{tra} is the number of sampled transitions.

In the testing stage, the trained policy MOQ network is employed to plan the UAV's trajectory of each time slot by network inference. Hence, based on the number of time slots and inference time of the policy MOQ network, MORL-TER's time complexity is $O(T \times (\sum_{i=1}^{L+1} \beta_{i-1} \times \beta_i))$ in the testing stage.

VI. SIMULATION RESULT AND DISCUSSION

We examine the optimization performance of the proposed MORL-TER by conducting extensive experiments. To facilitate this evaluation, we have developed a Python-based simulator using PyTorch 1.7, which helps us to thoroughly evaluate and analyze the performance of the MORL-TER across various experimental scenarios. In the simulation, we define several important experiment parameters. Assume that the number of time slots T is equal to 100. The duration of each time slot is considered as 3 seconds. Thus, we have five minutes for the UAV's mission period. We consider a square region and its side lengths, $x_{\max}$ and $y_{\max}$ are both restricted to 400 m. The UAV's flight velocity is set to 10 m/s. For a computation task, we set its

TABLE III
SIMULATION PARAMETERS

Parameter	Value
Value used in system model	
Collection mirror area (A)	10^{-2} m^2
Energy conversion efficiency of GSD (C^G)	0.9
Energy conversion efficiency of the UAV (C^U)	0.9
Initial laser beam size (D)	0.1 m
Computing capability of the UAV (f^U)	1 GHz
Channel power gain (g_0)	-30 dB
Floating height of the HAP (H)	20 km
Maximum task number the UAV can store ($N_{\max}$)	10
Laser transmission power of the HAP (P^H)	2000 W
Energy transmission power of the UAV (P^U)	1 W
Combined optical efficiency ($\mathcal{X}$)	0.2
Attenuation coefficient (η)	$10^{-6} / \text{m}$
Angular spread of laser beam (ϑ)	3.4×10^{-5}
Maximal azimuth angle ($\theta_{\max}$)	$\pi/4$
Effective capacitance coefficient (κ)	10^{-26} W
Value used in reinforcement learning	
Discount factor (γ)	0.995
Number of sampled transitions (N_{tra})	32
Number of sampled preferences ($N_{\mathbf{w}}$)	2
Maximum number of episodes ($N_{\max}^{\text{eps}}$)	2000

TABLE IV
TEST INSTANCES

Instance (M, U)	Number of GSDs (M)	Flight height (U)
I-(60,30)	60	30
I-(60,50)	60	50
I-(100,30)	100	30
I-(100,50)	100	50
I-(140,30)	140	30
I-(140,50)	140	50

$\mathcal{I}$ and $\mathcal{C}$ to 5 MB and 10 cycles/bit, respectively. The parameter of Bernoulli stochastic process for each GSD is randomly determined by selecting a value from set $\{0.3, 0.5, 0.7\}$. As introduced in Section V-B, we generate 150 evenly distributed weight vectors to mimic diverse objective preferences, i.e., Ω . A promising MORL algorithm should possess the capability to generalize across Ω while effectively adapting to changing preferences. The policy MOQ network has a two-layer full-connected neural network. Each layer of the network includes 64 neurons, utilizing rectified linear unit (ReLU) as the activation function. The Adam optimizer is employed with a learning rate of 0.001 for training the policy MOQ network. We adopt the decay rate 0.9995 to reduce the random exploration probability ε from 1 to 0.01. The balance weight ψ is initially set to 1, and it is decreased from 1 to 0.01. Table III lists other parameter values used in the experiment.

In order to fully evaluate the optimization performance, we generate various test instances using two important parameters, i.e., the number of GSDs, M , and the UAV's flight height, U . We specify $M \in \{60, 100, 140\}$ and $U \in \{30, 50\}$. Based on different combinations between M and U , we can generate six test instances to simulate different UAV-assisted MEC networks, as shown in Table IV.

TABLE V
RESULTS OF IGD

Instance (M, U)	NSGA-II	MOEA/D	Naive	MODQN	MORL-DW	MORL-TS	EMOQL	MORL-TER
I-(60,30)	0.2067	0.1957	0.1220	0.0602	0.0479	0.0485	0.0343	0.0056
I-(60,50)	0.3287	0.3819	0.1429	0.0989	0.0748	0.0558	0.0453	0.0064
I-(100,30)	0.1438	0.1272	0.0863	0.0716	0.0432	0.0552	0.0516	0.0059
I-(100,50)	0.1664	0.1168	0.0818	0.0598	0.0450	0.0427	0.0283	0.0126
I-(140,30)	0.1349	0.1273	0.1087	0.0921	0.0893	0.0853	0.0378	0.0111
I-(140,50)	0.1467	0.0915	0.0815	0.0724	0.0437	0.0343	0.0252	0.0098

TABLE VI
AVERAGE RANKS AND POSITIONS OF EIGHT ALGORITHMS BASED ON AER, AE, AND IGD

Algorithm	AER		AE		IGD	
	Average rank	Position	Average rank	Position	Average rank	Position
NSGA-II	N/A	N/A	N/A	N/A	7.8333	8
MOEA/D	N/A	N/A	N/A	N/A	7.1667	7
Naive	6.0000	6	5.0000	6	6.0000	6
MODQN	5.0000	5	4.8333	5	5.0000	5
MORL-DW	3.8333	4	4.0000	4	3.5000	4
MORL-TS	3.0000	3	2.6667	3	3.3333	3
EMOQL	2.0000	2	2.5000	2	2.1667	2
MORL-TER	1.0000	1	1.0000	1	1.0000	1

TABLE VII
AEE VALUES OF EIGHT ALGORITHMS

Instance (M, U)	NSGA-II	MOEA/D	Naive	MODQN	MORL-DW	MORL-TS	EMOQL	MORL-TER
I-(60,30)	4588.2968	4417.0441	4666.2617	4746.1316	4750.9198	4797.2326	4752.0688	4756.1426
I-(60,50)	4030.9909	3744.4423	3842.0415	4346.1316	4394.3907	4430.0549	4426.8519	4485.1426
I-(100,30)	3955.9134	3914.8485	4074.2826	4559.7300	4533.0856	4562.2499	4568.3572	4570.2803
I-(100,50)	3784.1530	3550.2815	3786.3759	3858.6753	3880.5634	3937.7893	4088.0439	4176.7917
I-(140,30)	3609.0996	3725.1822	3634.9195	4304.6842	4237.7060	4456.2557	4564.8346	4383.0884
I-(140,50)	3032.7680	2924.0091	3026.3812	3302.0170	3375.0501	3527.1929	3667.3174	3823.1062

TABLE VIII
ANT VALUES OF EIGHT ALGORITHMS

Instance (M, U)	NSGA-II	MOEA/D	Naive	MODQN	MORL-DW	MORL-TS	EMOQL	MORL-TER
I-(60,30)	39.2333	42.7667	43.1667	46.0733	39.6067	42.9000	65.4000	74.2867
I-(60,50)	40.2333	43.5667	94.9333	86.1467	91.3467	91.6800	99.7533	148.8133
I-(100,30)	60.1600	59.2800	61.2067	62.8467	63.3067	66.3867	66.1800	113.4600
I-(100,50)	134.6667	156.7200	133.1333	145.8267	146.8600	140.1867	163.9400	186.0133
I-(140,30)	72.1333	65.5133	69.6867	82.2333	87.1600	78.6333	99.3733	131.8333
I-(140,50)	135.3267	158.4667	143.6133	163.2733	183.2533	191.8200	205.6467	227.7933

TABLE IX
ACOI VALUES OF EIGHT ALGORITHMS

Instance (M, U)	NSGA2	MOEA/D	Naive	MODQN	MORL-DW	MORL-TS	EMOQL	MORL-TER
I-(60,30)	2302.2253	2405.7201	2472.7192	2484.5099	2496.4388	2501.3396	2536.4396	2542.6151
I-(60,50)	2201.2253	2385.7201	2419.4913	2355.1503	2461.9522	2468.8766	2469.3586	2496.7145
I-(100,30)	2280.6970	2387.8461	2375.7246	2373.4551	2470.2029	2482.1602	2477.5002	2487.9340
I-(100,50)	1877.9512	1975.5409	2016.5750	2356.4529	2358.7269	2359.4886	2365.9576	2397.8544
I-(140,30)	2243.0598	2216.3672	2244.6158	2337.1092	2408.7563	2457.0549	2460.9904	2479.8087
I-(140,50)	1602.0953	1762.5290	1638.4495	2139.0113	2213.3376	2231.9985	2256.4042	2358.4983

A. Evaluation Indicator

We adopt five widely used evaluation indicators to comprehensively measure the performance of MORL-TER. These indicators include the average episodic regret [36], adaption error [34], inverted generational distance [41], comprehensive objective indicator [2], and Friedman test [42].

1) *Average Episodic Regret*: We evaluate the performance of an MORL based on its regret. The regret is defined as the

difference between optimal and actual returns [36], as follows.

$$\begin{aligned} \Delta(\mathbf{w}, \mathbf{R}_1) &= \mathbf{w} \cdot \mathbf{V}_{\mathbf{w}}^* - \mathbf{w} \cdot \mathbf{R}_1 \\ &= \mathbf{w} \cdot \mathbf{V}_{\mathbf{w}}^* - \mathbf{w} \cdot \sum_{t=1}^T \gamma^{t-1} \mathbf{r}_t, \end{aligned} \quad (30)$$

where $\mathbf{V}_{\mathbf{w}}^*$ and $\mathbf{R}_1$ are the optimal and actual returns for preference $\mathbf{w}$ at time step $t = 1$, respectively. $\mathbf{r}_t$ is the immediate

TABLE X
AVERAGE RANKS AND POSITIONS OF EIGHT ALGORITHMS BASED ON AEE, ANT, AND ACOI

Algorithm	AEE		ANT		ACOI	
	Average rank	Position	Average rank	Position	Average rank	Position
NSGA-II	6.8333	8	7.3333	8	7.8333	8
MOEA/D	7.6667	7	6.3333	7	6.5000	7
Naive	6.5000	6	5.8333	6	6.0000	6
MODQN	4.6667	5	4.6667	5	5.6667	5
MORL-DW	4.3333	4	4.5000	4	4.0000	4
MORL-TS	2.3333	3	4.1667	3	2.8333	3
EMOQL	2.1667	2	2.6667	2	2.1667	2
MORL-TER	1.5000	1	1.0000	1	1.0000	1

vectorial rewards received at the t -th time step, $t = 1, \dots, T$. Differ from $\mathbf{R}_1$, the regret provides a shared $\mathbf{V}_w^*$ for preference $\mathbf{w}$. In other words, an optimal policy consistently yields a regret value of 0. Based on (30), the average episodic regret (AER) is calculated as

$$\text{AER} = \frac{1}{N_{\max}^{\text{eps}}} \sum_{i=1}^{N_{\max}^{\text{eps}}} \Delta(\mathbf{w}_i, \mathbf{R}_i), \mathbf{w}_i \in \Omega, \quad (31)$$

where $\mathbf{w}_i$, $\mathbf{R}_i$, and $\Delta(\mathbf{w}_i, \mathbf{R}_i)$ represent the preference, return, regret in the i -th episode, respectively. An MORL algorithm with a smaller AER has better performance.

2) *Adaptation Error*: To evaluate the adaptation capability of an MORL algorithm to preference dynamics, we compare $\mathbf{R}_1$ with $\mathbf{V}_w^*$ when a preference $\mathbf{w} \in \Omega$ is provided by the algorithm in the testing stage. The adaptation error (AE) is defined as the average relative error between $\mathbf{w} \cdot \mathbf{R}_1$ and $\mathbf{V}_w^*$:

$$\text{AE} = \frac{1}{\Phi} \sum_{\mathbf{w} \in \Omega} \left| \frac{\mathbf{w} \cdot \mathbf{R}_1 - \mathbf{w} \cdot \mathbf{V}_w^*}{\mathbf{w} \cdot \mathbf{V}_w^*} \right|. \quad (32)$$

A smaller AE represents the corresponding MORL is better adaptive to dynamic preferences.

3) *Inverted Generational Distance*: The Pareto front (PF) can be used to calculate the inverted generational distance (IGD). Let $\mathcal{F}_{\text{refer}}$ and $\mathcal{F}_{\text{known}}$ represent the referenced PF and known PF obtained by an algorithm, respectively. It is noted that $\mathcal{F}_{\text{refer}}$ may be unknown for highly complex MOO problems, such as the ETWC problem. In such cases, a widely used approach in the literature [2], [41], [42] is to collect the best solutions so far obtained by all algorithms and choose the non-dominated ones from them. The corresponding PF is treated as $\mathcal{F}_{\text{refer}}$. IGD is defined as

$$\text{IGD} = \frac{\sum_{\mu \in \mathcal{F}_{\text{refer}}} d(\mu, \mathcal{F}_{\text{known}})}{|\mathcal{F}_{\text{refer}}|}, \quad (33)$$

where $d(\mu, \mathcal{F}_{\text{known}})$ is the euclidean distance between μ in $\mathcal{F}_{\text{refer}}$ and its nearest point in $\mathcal{F}_{\text{known}}$. IGD can reflect both the convergence and diversity of a known PF. The smaller IGD value reflects better performance.

4) *Comprehensive Objective Indicator*: Given the ETWC problem with two objectives, we design a comprehensive objective indicator (COI), with E_{total} and N_{total} considered simultaneously. For each objective vector, we adopt linear scalarization to aggregate it into a COI. Note that two elements of an objective vector are corresponding to objectives E_{total} and N_{total} , respectively. After a single run, an MORL can achieve many

objective vectors for preference $\mathbf{w} = (w^1, w^2)$. Let $EN^i(\mathbf{w}) = (E_{\text{total}}^i(\mathbf{w}), N_{\text{total}}^i(\mathbf{w}))$ represent the i -th objective vector of $\mathbf{w}$, $i = 1, \dots, n_{\mathbf{w}}$. The COI of $EN^i(\mathbf{w})$ is defined as

$$\begin{aligned} \text{COI}^i(\mathbf{w}) &= \mathbf{w} \cdot EN^i(\mathbf{w}) \\ &= w^1 \cdot E_{\text{total}}^i(\mathbf{w}) + w^2 \cdot N_{\text{total}}^i(\mathbf{w}). \end{aligned} \quad (34)$$

The best objective vector for $\mathbf{w}$, $EN^b(\mathbf{w})$, is obtained by

$$EN^b(\mathbf{w}) = (E_{\text{total}}^b(\mathbf{w}), N_{\text{total}}^b(\mathbf{w})), b = \underset{i \in \{1, \dots, n_{\mathbf{w}}\}}{\text{argmax}} \text{COI}^i(\mathbf{w}), \quad (35)$$

where $E_{\text{total}}^b(\mathbf{w})$ and $N_{\text{total}}^b(\mathbf{w})$ are the best E_{total} and N_{total} for $\mathbf{w}$, respectively. Based on (34) and (35), we calculate the average energy efficiency (AEE), average number of tasks collected (ANT), and average COI (ACOI) for all preferences in Ω , defined as

$$\text{AEE} = \frac{1}{\Phi} \sum_{\mathbf{w} \in \Omega} E_{\text{total}}^b(\mathbf{w}), \quad (36)$$

$$\text{ANT} = \frac{1}{\Phi} \sum_{\mathbf{w} \in \Omega} N_{\text{total}}^b(\mathbf{w}), \quad (37)$$

$$\text{ACOI} = \frac{1}{\Phi} \sum_{\mathbf{w} \in \Omega} \text{COI}^b(\mathbf{w}). \quad (38)$$

5) *Friedman Test*: The test [42] is used to distinguish differences between algorithms in terms of algorithm-related metrics including AER, AE, and IGD, and system-related metrics including AEE, ANT, and ACOI. Consider all test instances, the average ranks of all algorithms are calculated. Based on these average ranks, the relative positions of each algorithm are determined, providing a clear indication of their respective performances.

B. Performance Evaluation

After running MORL-TER, we can obtain a single parametric policy MOQ network to generalize across the entire preference space. In other words, when providing a given preference, the trained policy MOQ network can output the corresponding optimal policy (i.e., flight trajectory) by simple algebraic calculations. Suppose the decision-maker's current preferences can be denoted as $\mathbf{w}_1 = (1.0, 0.0)$, $\mathbf{w}_2 = (0.0, 1.0)$, and $\mathbf{w}_3 = (0.5, 0.5)$. Preference $\mathbf{w}_1$ represents that one only focuses on maximizing the total energy efficiency E_{total} without considering objective N_{total} . Similarly, preferences $\mathbf{w}_2$ aims at maximizing the number of collected tasks, N_{total} , ignoring objective E_{total} .

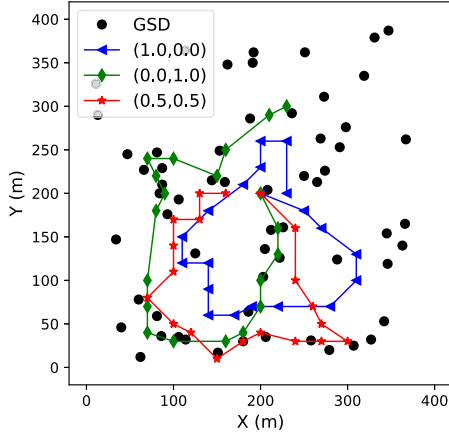


Fig. 4. Trajectories of the UAV under three different preferences.

Preference w_3 denotes that two objectives E_{total} and N_{total} are important equally, i.e., maximizing them simultaneously.

Fig. 4 shows the three trajectories corresponding to the above three preferences in I-(60,30). Note that the UAV's take-off point is set to the central point. The blue trajectory corresponds to w_1 that aims to maximize E_{total} , neglecting objective N_{total} . One can observe that the UAV operates in the sparsely populated GSD region, contributing to improving the UAV's energy efficiency. This is attributed to the fewer computation tasks collected, the lower transmitting energy consumption E_t^{Tr} and processing energy consumption E_t^{P} . The green trajectory is associated with w_2 that focuses on maximizing N_{total} individually. It is observed that the UAV moves to the dense GSD area to collect computation tasks as many as possible without considering energy efficiency. The red trajectory corresponds to w_3 that concentrates on maximizing E_{total} and N_{total} simultaneously. The UAV is observed to navigate within the dense GSD region to collect computation tasks. Nevertheless, compared with the blue trajectory, the UAV can avoid collecting excessive tasks because it needs to maintain appropriate energy efficiency. According to the analysis above, the proposed MORL-TER owns the capability to obtain potential control policies based on diverse preferences in a single run.

To fully investigate the performance of MORL-TER, we consider employing five state-of-the-art MORLs and compare the proposed approach with them in terms of AER, AE, IGD, COI, and Friedman test in six test instances. The compared algorithms are listed below.

- Naive: The approach [37] formulates an overall user utility through converting multiple single-objective Q-values into an aggregated function, which is used to select actions.
- MODQN: This approach is based on multi-objective deep Q-network [38], which aims to learn the vectorial Q-function, while the Q-network is only activated and learned on the current preference.
- MORL-DW: This approach combines the dynamic weights [36] and MORL, which aims to deal with dynamic preference problems. MORL-DW can adapt to changing preferences through learning and optimizing multi-objective Q-value vectors.

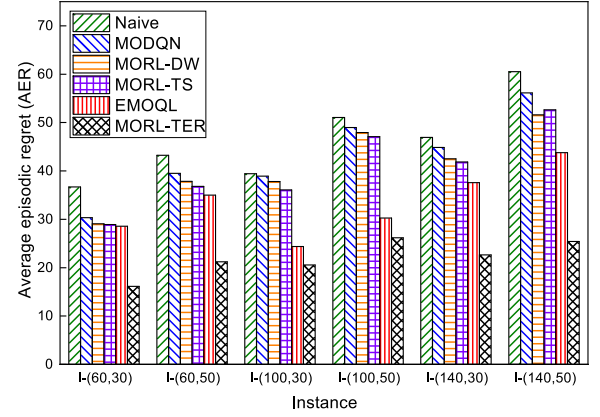


Fig. 5. Average episodic regret of six MORL algorithms.

- MORL-TS: This approach combines MORL-DW and the tournament selection scheme [39], to address the task offloading problem with dynamic preferences, aiming at minimizing completion time, energy consumption, and usage charge, simultaneously.
- EMOQL: This approach is introduced in [34], called envelope multi-objective Q-learning, which aims to learn a single parametric representation for optimal policies over the preference space.
- MORL-TER: The proposed EMOQL with the TER scheme in this paper.

Fig. 5 illustrates the AER values of the six MORL algorithms. It is obvious that MORL-TER outperforms the other five algorithms in all test instances since its AER is the smallest. Oppositely, the worst MORL algorithm is Naive because it has the largest AER values in all instances. Naive optimizes multi-objective Q-values separately and adopts an aggregated function to select actions. However, due to the independent updates of Q-values for individual objectives, its learned multi-objective Q-values may not effectively capture a favorable balance between objectives. Consequently, Naive cannot maximize two objectives, E_{total} and N_{total} , simultaneously. MODQN is identified as the second worst algorithm across all test instances. The primary reason for its poor performance is that its Q-network cannot incorporate the state and preference as its input. Moreover, the network is trained exclusively on the current preference. Thus, MODQN lacks the ability to retain learned policies for different preferences and fails to effectively adapt to varying preferences in dynamic MEC networks.

On the contrary, MORL-DW concatenates the current state and preference and feeds them into its Q-network, to adapt to the changing preferences. Moreover, the network is trained simultaneously on the current and encountered preferences, preventing it from overfitting to the current preference. Thus, MORL-DW outperforms MODQN regarding AER in almost all instances. However, MORL-DW lacks the ability to prioritize encountered preferences based on their significance, as it equally treats all preferences. Thus, this approach is prone to forgetting previously learned policies. By introducing the TS scheme, MORL-TS is better than MORL-DW in almost all instances. MORL-TS can

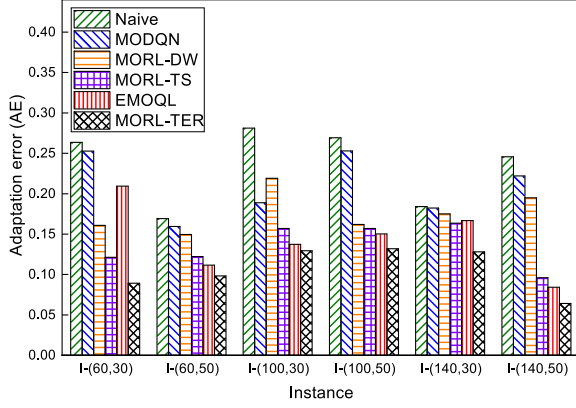


Fig. 6. Adaptation error of six MORL algorithms.

select those preferences that are more significant with higher probability, which enables its Q-network to memorize the previously learned optimal policies.

EMOQL surpasses MORL-TS in all test instances because it selects multiple preferences from the preference space to train the Q-network. Thus, EMOQL can obtain a reasonable coordination between preferences and corresponding optimal policies, preventing it from over-adapting to one of the preferences. This also demonstrates why EMOQL is selected for solving the ETWC problem. Nevertheless, EMOQL treats transitions as atomic units when considering them for addition in or deletion from the buffer, resulting in that the experiences obtained by a preference's optimal policy can be harmful to another preference's training. Unlike EMOQL, the proposed MORL-TER with the TER scheme treats traces (i.e., a trace consists of an episode's transitions) as atomic units when adding and deleting them from the experience buffer. The TER scheme adopts the crowding distance to reflect the relative diversity of a trace in the experience buffer $\mathcal{B}$. A trace with a bigger crowding distance has better diversity. Thus, the trace can be used to train another policy, so as to MORL-TER can maintain the previously learned optimal policies, and improve the MOO performance under dynamic preferences. This is why MORL-TER outperforms EMOQL in all test instances. Fig. 6 illustrates the adaptation error obtained by the six algorithms. It can be seen that MORL-TER exceeds the other five MORLs as it always has the smallest AE values. It means that MORL-TER can better deal with changing preferences in dynamic MEC networks than the other five algorithms.

We also show the cumulative weighted sum of the vectorial return (CWS-VR) in each episode, which can reflect an MORL's convergence performance and sampling efficiency. Based on the current preference $\mathbf{w}$ and vectorial return $\mathbf{R}_1$, we can calculate CWS-VR $\mathbf{w}\mathbf{R}_1$ obtained by an MORL at the end of each episode. Fig. 7 shows CWS-VR versus episode in I-(60,30), which reveals the convergence behaviors of six MORLs. One can observe that the proposed MORL-TER converges at a fast speed, for which around 1000 episodes are required. Moreover, MORL-TER has a faster convergence speed than the other five algorithms, which validates its effectiveness and availability.

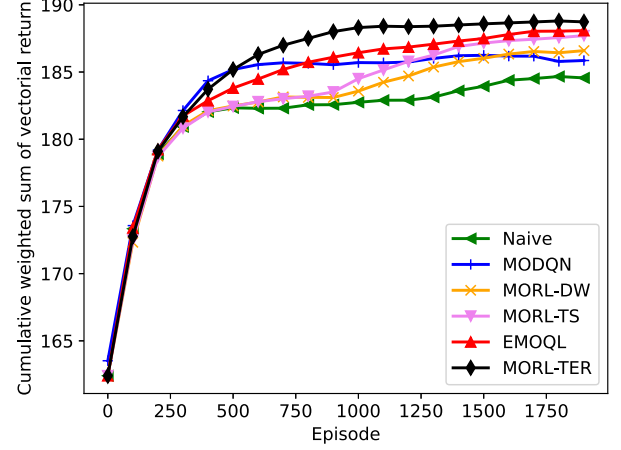
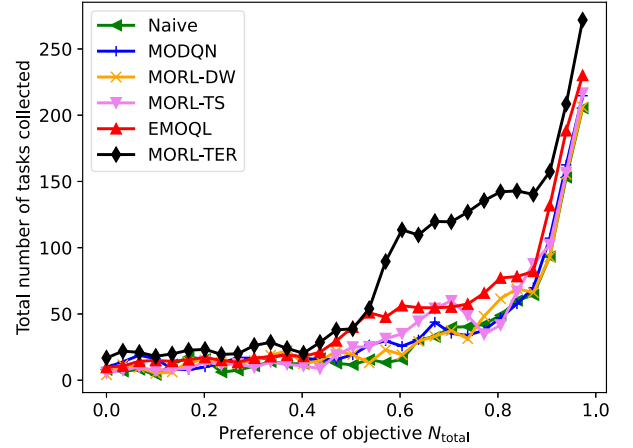


Fig. 7. Cumulative weighted sum of vectorial return versus episode.

Fig. 8. Total number of tasks collected versus preference of objective N_{total} .

In addition, Fig. 8 illustrates the total number of tasks collected versus the preference of objective N_{total} in I-(60,30). It is seen that the overall trend of task number is upward, as preferences of N_{total} increase. The larger preferences of N_{total} signify that the UAV pays more attention to collecting tasks. Furthermore, MORL-TER can obtain a larger number of tasks collected than the other five algorithms for almost all preferences of N_{total} . Thus, MORL-TER can collect more tasks by properly planning the UAV's flight trajectories.

We also add two well-known multi-objective evolutionary algorithms (MOEAs) into performance comparison, including NSGA-II and MOEA/D. The two algorithms are listed below.

- NSGA-II: The fast and elitist non-dominated sorting genetic algorithm is adopted to balance the task delay and energy consumption, aiming at meeting user requirements of diverse applications [43].
- MOEA/D: The multi-objective evolutionary algorithm based on decomposition is adopted to reduce the application delay and energy consumption in an MEC network [42].

IGD values obtained by eight algorithms are exhibited in Table V. IGD reflects the diversification and convergence of

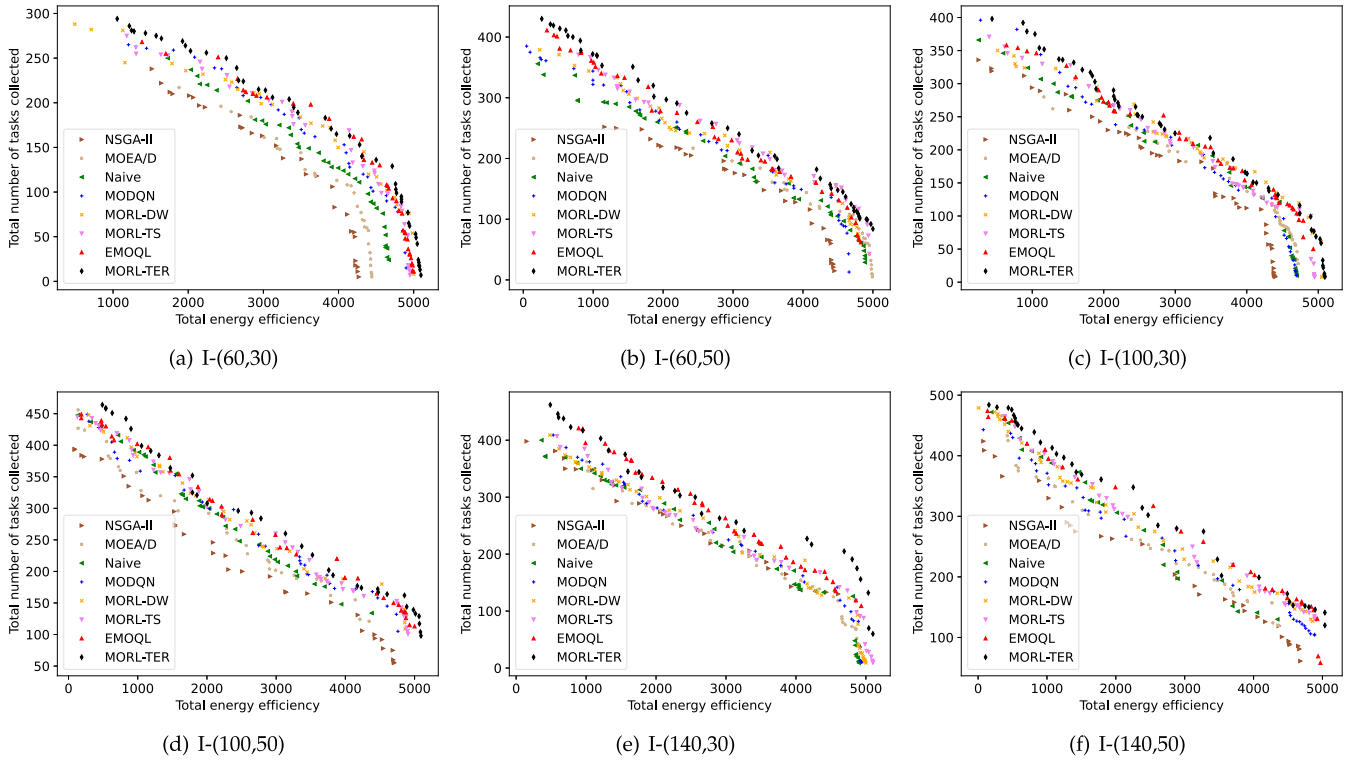


Fig. 9. Pareto fronts of eight algorithms.

multiple non-dominated policies simultaneously. At first glance, it is evident that two well-recognized algorithms, NSGA-II and MOEA/D, perform suboptimally, failing to discover satisfactory non-dominated policies in all test instances. The primary reason for this underperformance can be attributed to the inherent challenges posed by high-dimensional multi-objective optimization problems within dynamic environments, exemplified by the ETWC problem. MOEAs often expend a substantial amount of time in their pursuit of decent non-dominated policies, making it difficult to achieve convergence within tight time constraints. In particular, MOEAs with extensive encoding lengths, for example, 100, face a time-consuming process in generating acceptable non-dominated policies. Furthermore, the dynamic and uncertain nature of the UAV-assisted MEC environment compounds these challenges. In practical terms, these dynamics and uncertainties frequently necessitate the re-initiation of MOEAs from the beginning, imposing considerable computational overhead and diminishing their convergence speed.

Second, in all test instances, all MORLs consistently surpass NSGA-II and MOEA/D. A key distinction lies in the decision-making process: while MOEAs rely on a single chromosome to make decisions for all time slots, MORLs employ real-time decision-making for each time slot, taking into account the prevailing environmental conditions. Notably, MORLs seamlessly integrate reinforcement learning with deep neural networks, enabling them to tackle sequential decision-making challenges in dynamic MEC environments. This capability stems from their adaptability to environmental changes, achieved through iterative interactions with the MEC environment. Consequently,

MORLs exhibit rapid convergence and responsiveness to user requirements. These attributes render MORLs a superior choice for addressing the ETWC problem in comparison to MOEAs.

Finally, MORL-TER obtains the smallest IGD values in all test instances, validating its superiority over the other seven algorithms. MORL-TER can adapt to various user preferences by learning weight-dependent multi-objective Q-value vectors, thus it can obtain the optimal non-dominated policies corresponding to these preferences. To further support our observation above, the Pareto fronts of eight algorithms are shown in Fig. 9, which demonstrate the excellent performance of MORL-TER.

The Friedman test is employed to clearly reflect the position ranks of the eight algorithms with respect to AER, AE, and IGD. Based on the three metrics values, Table VI shows the average ranks and positions of the eight algorithms. It is clearly seen that MORL-TER achieves the best multi-objective optimization performance.

Tables VII and VIII illustrate the AEE and ANT values of eight algorithms, respectively, where the best results are in bold. Regardless of whether M or U is kept constant, the corresponding AEE values exhibit a tendency to decrease as the other parameter increases. First, if the number of GSDs grows, M , the UAV has to transmit more energy to them. Second, as the flight height increases, U , the coverage of the UAV expands correspondingly, hence it also transmits more energy to the GSDs within the coverage. However, transmitting more energy to GSDs results in lower energy efficiency. Table VII well supports this. On the other hand, no matter whether M or U is fixed, the associated ANT values increase as the other grows up. This is because either more GSDs or higher flight heights

will lead to more computation tasks being collected by the UAV. Table VIII also well supports this phenomenon.

In Table VII, it is observed that MORL-TER outperforms the other algorithms without regard to I-(60,30) and I-(140,30). While MORL-TS and EMOQL obtain decent AEE values in I-(60,30) and I-(140,30) respectively, they perform poorly in terms of ANT. For instance, although MORL-TS achieves the largest AEE value in I-(60,30), it results in a poor ANT value among all algorithms. While EMOQL obtains the optimal AEE value, its ANT value is worse than that of MORL-TER in I-(140,30). MORL-TER is the best because it obtains the largest ANT values in all test instances, as shown in Table VIII. This implies that the MORL-TER enables the UAV to efficiently collect abundant computation tasks from GSDs by appropriately optimizing the UAV's flight trajectories.

Table IX shows the ACOI values of eight algorithms. One can observe that although MORL-TER cannot acquire the best results in all test instances regarding AEE, it performs better than the other seven algorithms with respect to ACOI. As aforementioned, the ACOI can reflect the overall performance of a multi-objective optimization algorithm. Thus, MORL-TER can balance the two objectives E_{total} and N_{total} . Based on the results of AEE, ANT, and ACOI, Table X shows the average ranks and positions of eight algorithms. It is easily seen that MORL-TER achieves the best comprehensive performance.

VII. CONCLUSION

This paper models the problem of energy-efficient trajectory optimization with wireless charging (ETWC) by MOMDP and proposes a modified MORL algorithm with the trace-based experience replay scheme, namely MORL-TER, to solve the ETWC problem. MORL-TER can adapt to the dynamics of preferences, thus obtaining a tradeoff between objectives. Compared with five MORLs including Naive, MODQN, MORL-DW, MORL-TS, and EMOQL, and two MOEAs including NSGA-II and MOEA/D, our algorithm achieves better multi-objective optimization performance in all test instances regarding algorithm-related metrics, including average episodic regret, adaptation error, and inverted generational distance. MORL-TER also exhibits superior performance across various system-related metrics, such as the average energy efficiency, average number of tasks collected by the UAV, and average comprehensive objective indicator, in almost all instances. Furthermore, MORL-TER achieves the best rank in the Friedman test for both algorithm- and system-related metrics. Therefore, these results highlight excellent performance of MORL-TER and its potential applicability in UAV-assisted MEC networks characterized by dynamic preference requirements.

REFERENCES

- [1] F. Wang, J. Xu, and S. Cui, "Optimal energy allocation and task offloading policy for wireless powered mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 4, pp. 2443–2459, Apr. 2020.
- [2] F. Song et al., "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023, doi: [10.1109/TMC.2022.3208457](https://doi.org/10.1109/TMC.2022.3208457).
- [3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter, 2017.
- [4] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.
- [5] K. Zhang, X. Gui, D. Ren, and D. Li, "Energy-latency tradeoff for computation offloading in UAV-assisted multiaccess edge computing system," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6709–6719, Apr. 2021.
- [6] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.
- [7] Y. Liu, K. Xiong, Q. Ni, P. Fan, and K. B. Letaief, "UAV-assisted wireless powered cooperative mobile edge computing: Joint offloading, CPU control, and trajectory optimization," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2777–2790, Apr. 2020.
- [8] Y. Yu, J. Tang, J. Huang, X. Zhang, D. K. C. So, and K.-K. Wong, "Multi-objective optimization for UAV-assisted wireless powered IoT networks based on extended DDPG algorithm," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6361–6374, Sep. 2021.
- [9] Z. Cheng, M. Liwang, N. Chen, L. Huang, X. Du, and M. Guizani, "Deep reinforcement learning-based joint task and energy offloading in UAV-aided 6G intelligent edge networks," *Comput. Commun.*, vol. 192, pp. 234–244, Jun. 2022.
- [10] M. Zhao, Q. Shi, and M. Zhao, "Efficiency maximization for UAV-enabled mobile relaying systems with laser charging," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3257–3272, May 2020.
- [11] X. Hu, K.-K. Wong, and Y. Zhang, "Wireless-powered edge computing with cooperative UAV: Task, time scheduling and trajectory design," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 8083–8098, Dec. 2020.
- [12] K. Wang, X. Zhang, L. Duan, and J. Tie, "Multi-UAV cooperative trajectory for servicing dynamic demands and charging battery," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1599–1614, Mar. 2023.
- [13] M. Li, L. Liu, Y. Gu, Y. Ding, and L. Wang, "Minimizing energy consumption in wireless rechargeable UAV networks," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3522–3532, Mar. 2022.
- [14] Q. Liu et al., "Joint power and time allocation in energy harvesting of UAV operating system," *Comput. Commun.*, vol. 150, pp. 811–817, Jan. 2020.
- [15] J. Chen and J. Xie, "Joint task scheduling, routing, and charging for multi-UAV based mobile edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 1–6.
- [16] J. Shi, P. Cong, L. Zhao, X. Wang, S. Wan, and M. Guizani, "A two-stage strategy for UAV-enabled wireless power transfer in unknown environments," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1785–1802, Feb. 2024, doi: [10.1109/TMC.2023.3240763](https://doi.org/10.1109/TMC.2023.3240763).
- [17] J. Wang, C. Jin, Q. Tang, N. N. Xiong, and G. Srivastava, "Intelligent ubiquitous network accessibility for wireless-powered MEC in UAV-assisted B5G," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 2801–2813, Fourth Quarter, 2021.
- [18] Q. Li, L. Shi, Z. Zhang, and G. Zheng, "Resource allocation in UAV-enabled wireless-powered MEC networks with hybrid passive and active communications," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2574–2588, Feb. 2023.
- [19] Y. Zhang, Z. Na, H. Ren, and B. Lin, "Wireless-powered UAV-assisted MEC: Joint resource allocation and trajectory optimization," in *Proc. IEEE/CIC Int. Conf. Commun.*, 2023, pp. 1–6.
- [20] X. Gu, G. Zhang, M. Wang, W. Duan, M. Wen, and P.-H. Ho, "UAV-aided energy-efficient edge computing networks: Security offloading optimization," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4245–4258, Mar. 2022.
- [21] Z. Yang, W. Xu, and M. Shikh-Bahaei, "Energy efficient UAV communication with energy harvesting," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1913–1927, Feb. 2020.
- [22] L. Zhang, X. Liu, and N. Ansari, "UAV-assisted efficient far-field wireless charging for WSN," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 1734–1739.
- [23] Y. Li, S. Shi, Y. Miao, Z. Xu, C. Wu, and S. Zhang, "Energy efficiency optimization in multi-UAV energy harvesting network," in *Proc. IEEE Glob. Commun. Conf. Workshops*, 2022, pp. 1419–1424.
- [24] L. Zhang, A. Celik, S. Dang, and B. Shihada, "Energy-efficient trajectory optimization for UAV-assisted IoT networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4323–4337, Dec. 2022.
- [25] Z. Xiong et al., "UAV-assisted wireless energy and data transfer with deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 85–99, Mar. 2021.

- [26] S. Fu et al., "Energy-efficient UAV-enabled data collection via wireless charging: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 10209–10219, Jun. 2021.
- [27] K. Zhu et al., "Aerial refueling: Scheduling wireless energy charging for UAV enabled data collection," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1494–1510, Sep. 2022.
- [28] C. Chen, S. Gong, W. Zhang, Y. Zheng, and Y. C. Kiat, "Deep reinforcement learning based contract incentive for UAVs and energy harvest assisted computing," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 2224–2229.
- [29] L. Liu, K. Xiong, J. Cao, Y. Lu, P. Fan, and K. B. Letaief, "Average AoI minimization in UAV-assisted data collection with RF wireless power transfer: A deep reinforcement learning scheme," *IEEE Internet of Things J.*, vol. 9, no. 7, pp. 5216–5228, Apr. 2022.
- [30] O. S. Oubbati, A. Lakas, and M. Guizani, "Multiagent deep reinforcement learning for wireless-powered UAV networks," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16044–16059, Sep. 2022.
- [31] S. Zhang and R. Cao, "Multi-objective optimization for UAV-enabled wireless powered IoT networks: An LSTM-based deep reinforcement learning approach," *IEEE Commun. Lett.*, vol. 26, no. 12, pp. 3019–3023, Dec. 2022.
- [32] A. M. Seid, J. Lu, H. N. Abishu, and T. A. Ayall, "Blockchain-enabled task offloading with energy harvesting in multi-UAV-assisted IoT networks: A multi-agent DRL approach," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3517–3532, Dec. 2022.
- [33] M. Fan et al., "Deep reinforcement learning for UAV routing in the presence of multiple charging stations," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 5732–5746, May 2023.
- [34] R. Yang, X. Sun, and K. Narasimhan, "A generalized algorithm for multi-objective reinforcement learning and policy adaptation," in *Proc. Annu. Conf. Neural Inform. Proc. Syst.*, 2019, pp. 1–12.
- [35] X. Chen et al., "Information freshness-aware task offloading in air-ground integrated edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 243–258, Jan. 2022.
- [36] A. Abels, D. M. Roijers, T. Lenaerts, A. Now'e, and D. Steckelmacher, "Dynamic weights in multi-objective deep reinforcement learning," in *Proc. ACM Int. Conf. Mach. Learn.*, 2019, pp. 1–10.
- [37] C. Liu, X. Xu, and D. Hu, "Multiobjective reinforcement learning: A comprehensive overview," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 45, no. 3, pp. 385–398, Mar. 2015.
- [38] H. Mossalam, Y. M. Assael, D. M. Roijers, and S. Whiteson, "Multi-objective deep reinforcement learning," 2016, *arXiv:1610.02707*. [Online]. Available: <http://arxiv.org/abs/1610.02707>
- [39] F. Song, H. Xing, X. Wang, S. Luo, P. Dai, and K. Li, "Offloading dependent tasks in multi-access edge computing: A multi-objective reinforcement learning approach," *Future Gener. Comput. Syst.*, vol. 128, pp. 333–348, Mar. 2022.
- [40] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, Aug. 2014.
- [41] W. Xu, C. Chen, S. Ding, and P. M. Pardalos, "A bi-objective dynamic collaborative task assignment under uncertainty using modified MOEA/D with heuristic initialization," *Expert Syst. Appl.*, vol. 140, pp. 1–24, Feb. 2020.
- [42] F. Song, H. Xing, S. Luo, D. Zhan, P. Dai, and R. Qu, "A multiobjective computation offloading algorithm for mobile-edge computing," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8780–8799, Sep. 2020.
- [43] L. Cui et al., "Joint optimization of energy consumption and latency in mobile edge computing for Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4791–4803, Jun. 2019.



Fuhong Song received the MEng degree in computer technology and the PhD degree in computer science and technology from Southwest Jiaotong University, Chengdu, China, in 2018 and 2022, respectively. He is currently a lecturer with the School of Information, Guizhou University of Finance and Economics. His research interests include mobile edge computing, multi-objective optimization, and reinforcement learning.



Mingsen Deng (Member, IEEE) is a professor of computer science with the School of Information, Guizhou University of Finance and Economics in China. He has published more the 100 papers in prestigious journals and distinguished international conferences. He has been an executive member of Technical Committee of High-Performance Computing of China Computer Federation since 2010, and received numerous awards, including Outstanding Scientists and Technologists of the Chinese Institute of Electronics (2020), One Hundred Person Project of the Guizhou Province (2016), Young Scientist Award of Guizhou Province (2018). His research interests focus on parallel and distributed computing, electronic structure calculations, and network analysis for Big-Data.



Huanlai Xing (Member, IEEE) received the PhD degree in computer science from the University of Nottingham (Supervisor: Dr Rong Qu), Nottingham, U.K., in 2013. He was a visiting scholar in computer science with the University of Rhode Island (Supervisor: Dr. Haibo He), USA, in 2020–2021. He is with the School of Computing and Artificial Intelligence, Southwest Jiaotong University (SWJTU), and Tangshan Institute of SWJTU. He was on editorial board of the *Science China Information Sciences*. He was a member of several international conference program and senior program committees, such as ECML-PKDD, MobiMedia, ISCIT, ICC, TrustCom, IJCNN, and ICSINC. His research interests include semantic communication, representation learning, data mining, reinforcement learning, machine learning, network function virtualization, and software defined networking.



Yanping Liu received the BE degree in electronic and information engineering and the ME degree in communication engineering from Chongqing University, Chongqing, China, in 2006 and 2009, and the PhD degree in information and communication engineering from Southwest Jiaotong University, Chengdu, China, in 2018. He is currently an associate professor with the School of Big Data Statistics, Guizhou University of Finance and Economics. His research interests include game theory, learning theory, and optimization theory for the resource management on UAV communication, and mmWave communication.



Fei Ye received the bachelor's degree from the Chengdu University of Technology, China, in 2014, and the master's degree in computer science and technology from Southwest Jiaotong University, China, in 2018. He is currently working toward the PhD degree in computer science with the University of York. His research topics includes deep generative image models, lifelong learning, and mixture models.



Zhiwen Xiao received the BEng degree in network engineering from the Chengdu University of Information Technology, Chengdu, China, in 2019, and the MEng degree in computer science from the Northwest A & F University, Yangling, China, in 2023. He is currently working toward the PhD degree in computer science with Southwest Jiaotong University, Chengdu, China. His research interests include semantic communication, federated learning, representation learning, data mining, and computer vision.